



University of
St Andrews

**Machine Learning Methods for
Information Transfer Quantification**

Chedy Smaoui

School of Mathematics and Statistics
University of St Andrews

Supervised by Dr Giorgos Minas

I certify that this project report has been written by me, is a record of work carried out by me, and is essentially different from work undertaken for any other purpose or assessment.

Signed: Chedy

Date: April 24, 2026

Abstract

As Francis Crick wrote, “The central dogma of molecular biology deals with the detailed residue-by-residue transfer of sequential information” (Crick, 1970). More broadly, biology depends on information transfer at many levels, from DNA to proteins, from the environment to the cell, and between cells. Mutual information (MI) provides a natural way to quantify this transfer in cell signalling, by measuring how much an observed intracellular response reveals about the stimulus that produced it. Estimating mutual information in practice is difficult, however, because biological responses are often high-dimensional, time-dependent, and only observed through limited data. In this project, we compare several machine learning methods for estimating mutual information in this setting. The probability-based framework of Statistical Learning Estimation of Mutual Information (SLEMI) and the decoding-based confusion-matrix approach both use a classifier and were both designed with this biological setting in mind, but neural methods such as Mutual Information Neural Estimation (MINE) and Vector Copula Estimation (VCE) were developed as more general-purpose methods. To compare all four approaches under controlled conditions, I use synthetic stimulus-response trajectories generated from simple class-dependent autoregressive models with both Gaussian and Student- t innovations, so that the true mutual information is known up to high numerical accuracy. The results show that no single method is best in every setting. MINE performs best for large sample sizes and in the most difficult settings, where the mutual information is low and the response dimension is high. For smaller sample sizes and weaker information transfer, the decoding-based estimator is more robust than SLEMI. VCE performs well in some low-dimensional, low-MI settings, but is less consistent overall. These results suggest that neural estimators offer a real advantage in the hardest estimation problems, whereas classifier-based methods remain competitive, and often more practical, when data are limited or the problem is less demanding.

Contents

1	Introduction	3
1.1	Aims and Objectives	5
1.2	Readership and Prerequisites	5
2	Background	6
2.1	Entropy and Differential Entropy	6
2.2	Kullback-Leibler Divergence	7
2.3	Mutual Information	7
3	Probability-Based Estimation	9
3.1	The Variational Bound	9
3.2	From Bound to Estimator	10
4	Decoding-Based Estimation	12
4.1	The Confusion Matrix Approach	12
4.2	The Data Processing Inequality	13
5	Neural Variational Estimation: MINE	14
5.1	The Donsker-Varadhan Representation	14
5.2	The MINE Estimator	16
6	Vector Copula Estimation (VCE)	18
6.1	Vector Copula Theory	18
6.2	Learning The Marginal Learning with Flow Matching	20
6.3	Computing Vector Ranks	21
6.4	Estimating the Copula Density	22
7	Experimental Comparison	24
7.1	Simulation Setup	24
7.2	Results	28
8	Conclusion	33
A	Mutual Information as Vector Copula Entropy	35

1 Introduction

Cells constantly receive signals from their surroundings and must respond appropriately in order to survive. In biology, this can be studied as a problem of information transfer, where an external condition acts on the cell and we record an intracellular quantity that reflects the cell’s response. [Tang and Hoffmann \(2022\)](#) describe this using the language of information theory, where the sender is usually an environmental stimulus and the receiver is a measured intracellular response. In this project, I use S to denote the stimulus and X to denote the measured cellular response.

A concrete example helps make this clearer. In the yeast study of [Granados et al. \(2018\)](#), cells are first grown in a normal, nutrient-rich environment and then suddenly exposed to different kinds of stress, such as low glucose, osmotic stress, or oxidative stress. In that setting, the stimulus S tells us which external condition the cell is experiencing. Because the experiment chooses from a fixed set of conditions, S is a discrete variable. The response X is not the full internal state of the cell, which would be far too complex to observe directly, but a measured intracellular readout of that state. In this case, X can be taken as the time series of how much of a transcription factor is found in the nucleus of a single cell, measured at many time points before and after stress is applied. Put simply, S is the external signal presented to the cell, and X is the corresponding intracellular measurement that we observe, as shown in Figure 1.

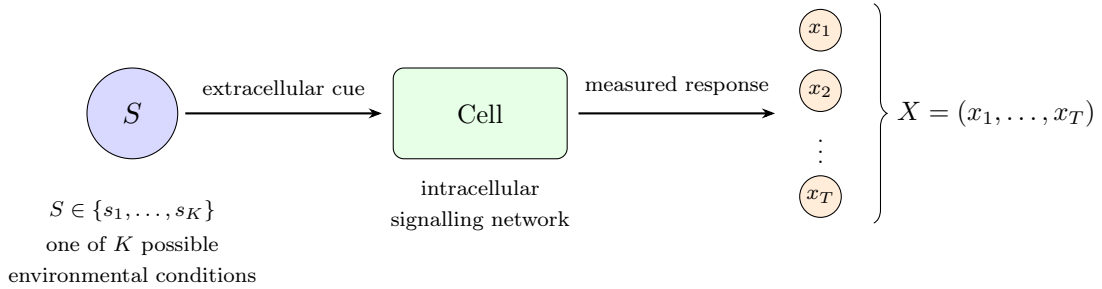


Figure 1: A discrete environmental stimulus $S \in \{s_1, \dots, s_K\}$ produces a time-dependent cellular response $X \in \mathbb{R}^T$.

The central quantity we want to study is the mutual information between the stimulus S and the response X . In this setting, mutual information measures how much observing the cellular response reduces uncertainty about which stimulus was applied. Following [Tang and Hoffmann \(2022\)](#), if S is the set of experimental stimulus conditions and X is the measured signalling response, then $I(X; S)$ quantifies how much information the intracellular response carries about stimulus identity and dose. Hence, if $I(X; S)$ is small, then responses produced by different stimuli overlap strongly and are hard to tell apart. By contrast, if $I(X; S)$ is large, then the responses are more distinguishable, meaning that the cell’s signalling activity carries clearer information about the outside condition.

This becomes particularly interesting when X is not just a single measurement, but a whole trajectory over time. In many signalling experiments, the response is recorded at many time points, so each cell is represented by a time series rather than one number. This is important because signalling dynamics can themselves encode information: two stimuli may produce responses with the same value at a particular time point, even though one response remains approximately constant while the other oscillates. In that case, a single snapshot may fail to distinguish the

two stimuli, whereas the full trajectory can. The yeast experiments of [Granados et al. \(2018\)](#) illustrate this clearly, showing that information available from the full time series can be greater than the information available from any single snapshot.

However, estimating mutual information from experimental data is difficult because it depends on the conditional distribution of the response under each stimulus, and these distributions are usually unknown. In real biological experiments, the response is often a high-dimensional time series observed only through limited data, which makes the relevant probability distributions hard to estimate directly. To compare different machine learning methods under controlled conditions, I therefore use synthetic time-series data generated from a known model. In that setting, the mutual information can be written as

$$I(S; X) = \mathbb{E} \left[\log \frac{p(X | S)}{p(X)} \right],$$

so once the data-generating model is specified, it can be approximated numerically by averaging this quantity over a very large simulated sample. This provides a reference value against which the estimation methods can be compared.

Here, the stimulus S is a discrete variable representing one of several possible environmental conditions, while the response $X = (X_1, \dots, X_T)$ is a stochastic trajectory observed at T time points as in Figure 1. To generate the synthetic trajectories, I used a simple autoregressive AR(1) model ([Brockwell and Davis, 2016](#)) and let its parameters depend on the stimulus class. AR(1) models are a standard way of representing time-dependent processes, while the particular choices of μ_k , α_k and τ_k here are chosen as part of my own simulation setup. More precisely, for a stimulus k , I generated the response according to

$$X_t = \alpha_k X_{t-1} + (1 - \alpha_k) \mu_k + \varepsilon_t, \quad t = 1, \dots, T, \quad (1)$$

with $X_0 = 0$, where μ_k controls the typical response level, α_k controls the persistence over time, and ε_t is the innovation term. I considered two different distributions for this innovation term which is the part of the model that adds randomness at each time step. It represents small unpredictable fluctuations in the response, so that even trajectories generated under the same stimulus do not look exactly the same. First, I used Gaussian innovations,

$$\varepsilon_t \sim \mathcal{N}(0, \tau_k^2)$$

so the noise is centred around zero and large deviations are relatively uncommon. Second, I used Student- t innovations with $\nu = 3$ degrees of freedom,

$$\varepsilon_t = \tau_k \sqrt{\frac{\nu - 2}{\nu}} Z_t, \quad Z_t \sim t_\nu$$

which are also centred around zero, but they have heavier tails than a Gaussian distribution, meaning that unusually large jumps can occur more often. To make the two settings easier to compare, I rescaled the Student- t innovations so that they have the same variance τ_k^2 as in the Gaussian case. By doing this, the main difference between the two models is not the overall amount of noise, but the fact that the Student- t case allows more extreme fluctuations.

The probability-based framework of [Jetka et al. \(2019\)](#) and the decoding-based approach of [Granados et al. \(2018\)](#); [Cepeda-Humerez et al. \(2019\)](#) were designed with this biological setting in

mind, where a discrete stimulus produces noisy temporal responses and the mutual information is typically low. We extend this line of work by adding two neural estimators, the Mutual Information Neural Estimator (MINE) (Belghazi et al., 2018) and the Vector Copula Estimator (VCE) (Chen et al., 2025), which were originally developed and tested on continuous high-dimensional problems with potentially large mutual information values. We are interested in how these methods apply to a cell signalling setup, where the stimulus is discrete, the response is a time-series trajectory, and the true MI is of order 1-2 nats or less. A central question of this project is therefore whether these general-purpose neural estimators offer any advantage over the specialised classifier-based methods in this setting, or whether their added complexity is unnecessary when the problem is inherently low-dimensional and low-information.

1.1 Aims and Objectives

The aim of this project is to compare several machine learning methods for estimating mutual information in biological cell-signalling settings, with particular emphasis on the contrast between methods developed for this application and more general-purpose estimators.

The objectives of the project are:

- to implement four mutual-information estimators and compare them on the same synthetic signalling dataset;
- to evaluate how their performance changes with the number of time steps and the separation between stimulus classes;
- to assess the relative strengths and limitations of classifier-based and neural estimators in biological cell-signalling problems with low-information, time-dependent settings;
- to determine when the additional flexibility of general-purpose neural estimators leads to better performance, and when simpler classifier-based methods remain competitive;

1.2 Readership and Prerequisites

This project is primarily intended for final-year undergraduate students in mathematics and statistics. It assumes a solid foundation in probability theory and multivariate statistics. Specifically, the reader is expected to be familiar with continuous and discrete random variables, together with their associated probability density and mass functions, as well as with fundamental statistical quantities such as expectation $\mathbb{E}[\cdot]$, variance $\text{Var}(\cdot)$, and covariance matrices. A basic understanding of statistical inference is also assumed. No prior knowledge of Information Theory is assumed. Fundamental concepts such as Shannon Entropy, Differential Entropy, Kullback-Leibler divergence (D_{KL}), and Mutual Information (I) will be defined from first principles in Section 2.

Some background in statistical machine learning is also useful, in particular familiarity with standard classification methods such as Quadratic Discriminant Analysis (QDA), Support Vector Machines (SVM), and Random Forests. No prior knowledge of neural networks is assumed, and other machine learning ideas are explained as they arise.

2 Background

Before comparing different estimation methods, we need to cover some concepts from information theory. The main quantity we want to study is the mutual information between the stimulus S and the response X , but this is built from a few simpler ideas first. We therefore start with entropy, which measures uncertainty, and then move to the Kullback-Leibler divergence, which measures how different two probability distributions are. These two concepts lead us to the definition of mutual information.

2.1 Entropy and Differential Entropy

Entropy measures the uncertainty of a random variable before it is observed. If one or a few outcomes have most of the probability, the uncertainty is low and so is the entropy. If the probability is spread more evenly across many possible outcomes, the uncertainty is higher and so is the entropy. For a discrete random variable S taking values in $\mathcal{S} = \{s_1, \dots, s_K\}$, with probability mass function $P(s)$ for $s \in \mathcal{S}$, the Shannon entropy $H(S)$ (Shannon, 1948) is defined by

$$H(S) = - \sum_{s \in \mathcal{S}} P(s) \log P(s). \quad (2)$$

When the random variable is continuous rather than discrete, the corresponding quantity is the differential entropy. If X takes values in $\mathcal{X}$ and has probability density function $p(x)$ on $\mathcal{X}$, then

$$h(X) = - \int_{\mathcal{X}} p(x) \log p(x) dx. \quad (3)$$

Here, X does not have to be a scalar random variable, but can also be a vector-valued response, as in this project where $X = (X_1, \dots, X_T)$ is a trajectory in $\mathbb{R}^T$.

Differential entropy plays a role analogous to Shannon entropy in the continuous setting, in that it measures uncertainty in the continuous setting. However, it is not interpreted in exactly the same way as in the discrete case, because it can take negative values. This happens when the density is highly concentrated, so that $p(x)$ is greater than 1 on a sufficiently small region. For example, if $X \sim \text{Uniform}(0, a)$, then

$$h(X) = \log a,$$

which is negative when $a < 1$. This is not a contradiction: unlike probabilities, a probability density is not itself constrained to lie between 0 and 1, because only its integral over a region must equal probability. Thus, differential entropy is still useful as a measure of uncertainty or spread, but its absolute value does not have quite the same direct interpretation as in the discrete case.

We will also need joint and conditional versions of entropy. If S and X are discrete random variables, their joint entropy is

$$H(S, X) = - \sum_{s \in \mathcal{S}} \sum_{x \in \mathcal{X}} P(s, x) \log P(s, x), \quad (4)$$

and the conditional entropy of S given X is

$$H(S | X) = - \sum_{x \in \mathcal{X}} P(x) \sum_{s \in \mathcal{S}} P(s | x) \log P(s | x). \quad (5)$$

The conditional entropy measures how much uncertainty about S remains after observing X . In the mixed setting relevant for this project, where S is discrete and X is continuous, the same idea applies, except that sums over X are replaced by integrals.

2.2 Kullback-Leibler Divergence

So far, we have used entropy to describe uncertainty in random variables and their relationships. To study information transfer, however, we also need a way to compare probability distributions directly. This is where relative entropy, also known as the Kullback-Leibler divergence, comes in. For two probability distributions P and Q with the same domain, the Kullback-Leibler divergence from Q to P is defined as (Cover and Thomas, 1991)

$$D_{\text{KL}}(P\|Q) = \sum_{x \in \mathcal{X}} P(x) \log \frac{P(x)}{Q(x)} \quad (6)$$

in the discrete case, and

$$D_{\text{KL}}(P\|Q) = \int_{\mathcal{X}} p(x) \log \frac{p(x)}{q(x)} dx \quad (7)$$

in the continuous case. Here we assume that whenever $Q(x) = 0$ in the discrete case, we also have $P(x) = 0$, and similarly that whenever $q(x) = 0$ in the continuous case, we also have $p(x) = 0$, so that the logarithm is well defined. As usual, terms of the form $0 \log(0/0)$ are taken to be 0. Equivalently, this can be written as

$$D_{\text{KL}}(P\|Q) = \mathbb{E}_P \left[\log \frac{p(X)}{q(X)} \right]. \quad (8)$$

which shows more clearly what the KL divergence is doing. It says that we sample X from the true distribution p and then look at the log ratio $\log \frac{p(X)}{q(X)}$. If the outcomes that are common under p are also assigned similar probability by q , then this quantity stays small. But if q gives low probability to outcomes that often occur under p , then the KL divergence becomes large. So, the KL divergence tells us how different q is from p when we assume that the data come from p .

An important property, as noted in Cover and Thomas (1991), is that the relative entropy is always non-negative ($D_{\text{KL}}(p\|q) \geq 0$) and is zero if and only if $p = q$. However, it is not a true metric because it is not symmetric and does not satisfy the triangle inequality. Indeed, in general $D_{\text{KL}}(P\|Q) \neq D_{\text{KL}}(Q\|P)$.

2.3 Mutual Information

We are now ready to define mutual information, which measures the reduction in uncertainty by comparing the joint distribution of the two variables with what their distributions would look like if they were independent. For two random variables S and X , the mutual information is defined as the KL divergence between their joint distribution and the product of their marginal distributions:

$$I(S; X) = D_{\text{KL}}(p(s, x) \| p(s)p(x)) \quad (9)$$

The numerical value of mutual information depends on the base of the logarithm used in the Kullback-Leibler divergence (definition (8)). Throughout this report, logarithms are natural unless stated otherwise, so mutual information is measured in nats. If base-2 logarithms are used instead, the corresponding unit is bits. Notice that if S and X are independent, then $p(s, x) = p(s)p(x)$,

and the KL divergence is zero and therefore $I(S; X) = 0$. In that case, observing X tells us nothing about S . However, if the response depends strongly on the stimulus, then the joint distribution looks quite different from the product of the marginals, and the mutual information becomes larger. So, mutual information measures the strength of the dependence between the two variables.

In our setup from Figure 1, the set of experimental conditions S is discrete, while the measured signalling trajectory X is continuous and often multivariate. In this mixed discrete-continuous case, we write the mutual information as

$$\begin{aligned}
I(S; X) &= \sum_{i=1}^K \int_{\mathbb{R}^d} p(s_i, x) \log \frac{p(s_i, x)}{P(s_i) p(x)} dx && \text{(Definition 9)} \\
&= \sum_{i=1}^K \int_{\mathbb{R}^d} P(s_i) p(x|S = s_i) \log \frac{P(s_i) p(x|S = s_i)}{P(s_i) p(x)} dx && \text{(Chain rule)} \\
&= \sum_{i=1}^K P(s_i) \int_{\mathbb{R}^d} p(x|S = s_i) \log \frac{p(x|S = s_i)}{p(x)} dx && (10)
\end{aligned}$$

Using Bayes' rule, this can also be rewritten in terms of the conditional probability of the stimulus given the response:

$$I(S; X) = \sum_{i=1}^K P(s_i) \int_{\mathbb{R}^d} p(x | S = s_i) \log \frac{p(s_i | x)}{P(s_i)} dx. \quad (11)$$

This form is useful because it shows that mutual information depends on how much observing a response x changes the probability we assign to each possible stimulus. If $p(s_i | x)$ is close to the prior probability $P(s_i)$, then observing x tells us little about the stimulus. If $p(s_i | x)$ differs substantially from $P(s_i)$, then the response is informative about the stimulus.

For each stimulus level s_i , there is a corresponding distribution of cellular responses, and mutual information measures how informative these response distributions are about stimulus identity. If the conditional distributions $p(x | s_i)$ overlap heavily, then the different stimuli produce similar responses and the mutual information is small. If they are well separated, then the responses are easier to tell apart and the mutual information is larger.

Another important way to write mutual information is in terms of entropy ([Cover and Thomas, 1991](#), Theorem 2.4.1):

$$I(S; X) = H(S) - H(S|X), \quad (12)$$

where, in the present setting with discrete S and continuous X , the conditional entropy is

$$H(S | X) = - \int_{\mathbb{R}^d} p(x) \sum_{i=1}^K p(s_i | x) \log p(s_i | x) dx. \quad (13)$$

This shows that mutual information is the difference between how uncertain we are about the stimulus before seeing the response, $H(S)$, and the uncertainty that remains after observing it, $H(S | X)$.

3 Probability-Based Estimation

In the next two sections, we present two approaches to estimate mutual information that both begin in the same way. Given a dataset of labelled stimulus-response pairs $\{(s_i, x_i)\}_{i=1}^n$, where each response trajectory class x_i is paired with the stimulus class s_i that produced it, we train a classifier to predict the stimulus from the response. When we then give this classifier a new trajectory x it outputs a vector of estimated probabilities $\hat{p}(s|x)$, one entry for each stimulus class, representing how likely the classifier thinks it is that x came from each class.

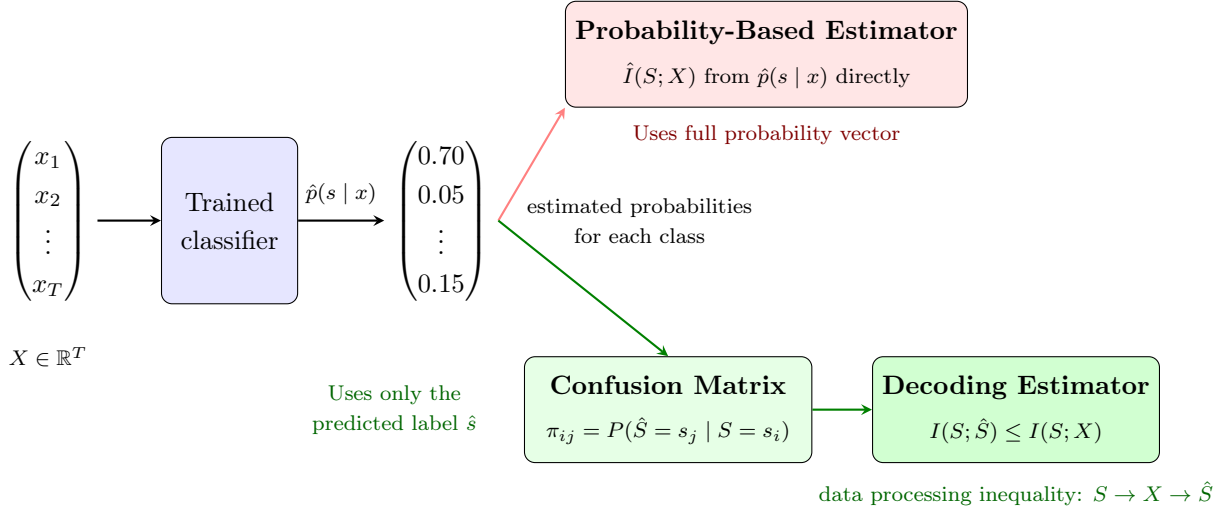


Figure 2: Classifier-based mutual information estimation. The classifier output $\hat{p}(s|x)$ is used in two ways: the probability-based estimator uses the full predicted probability vector, while the decoding-based estimator uses only the predicted label $\hat{s}$ via the confusion matrix.

The two approaches differ in what we do with this output, as illustrated in Figure 2. The probability-based estimator of [Jetka et al. \(2019\)](#), which we describe in this section, uses the full probability vector directly and plugs it into a formula derived from a variational bound on the mutual information. The decoding-based estimator of [Cepeda-Humerez et al. \(2019\)](#), which we cover in Section 4, discards the probabilities and keeps only the most likely class. We then compare these hard predictions against the true labels to build a confusion matrix, from which a separate MI estimate can be computed. Both approaches produce lower bounds on the true mutual information $I(S; X)$, but they do so for different reasons and with different trade-offs, as we shall see.

3.1 The Variational Bound

Looking at equation (12), we notice that the entropy term $H(S)$ is known since we control the stimulus distribution in our experiments. Therefore, the problem of estimating $I(S; X)$ reduces to estimating the conditional entropy term $H(S | X)$. In the present mixed discrete-continuous setting,

$$H(S | X) = -\mathbb{E}[\log p(s | x)], \quad (14)$$

where the expectation is over the joint distribution $p(s, x)$. Here, $p(s | x)$ is the conditional probability of stimulus s given response x . If this quantity were known exactly, then mutual information could be evaluated directly from equation (12). In practice, however, $p(s | x)$ is

unknown and must be approximated. This is where classifiers come in: a classifier trained to predict S from X is learning exactly this kind of approximation. In this way, the mutual information estimation problem can be recast as a supervised learning problem.

Now, following [Barber and Agakov \(2003\)](#), suppose that we have some approximation $q(s | x)$ to the true conditional probability $p(s | x)$. The source of this approximation is not important for the argument. For each fixed response x , both $p(s | x)$ and $q(s | x)$ are probability distributions over the possible stimuli, so

$$\begin{aligned} D_{\text{KL}}(p(s | x) \| q(s | x)) &= \sum_{s \in \mathcal{S}} p(s | x) \log \frac{p(s | x)}{q(s | x)} \\ &= \sum_{s \in \mathcal{S}} p(s | x) \log p(s | x) - \sum_{s \in \mathcal{S}} p(s | x) \log q(s | x). \end{aligned} \quad (15)$$

Since the Kullback-Leibler divergence is always non-negative, it follows that

$$\sum_{s \in \mathcal{S}} p(s | x) \log p(s | x) - \sum_{s \in \mathcal{S}} p(s | x) \log q(s | x) \geq 0 \quad (16)$$

for every fixed x . We now average this inequality over the distribution of the response X , that is, over $p(x)$. This gives

$$-\mathbb{E}[\log q(s | x)] \geq H(S | X), \quad (17)$$

where the expectation is over the joint distribution of (S, X) . Thus, replacing the true conditional probability $p(s | x)$ by an approximation $q(s | x)$ can only increase the conditional entropy term. Substituting this into equation (12) yields

$$I(S; X) \geq H(S) + \mathbb{E}[\log q(s | x)], \quad (18)$$

which is the Barber-Agakov variational lower bound. This lower bound holds for any choice of $q(s | x)$, and it becomes an equality when $q(s | x) = p(s | x)$. However, in practice, the expectation is replaced by an empirical average over a finite sample, so because of sampling variability, it may overestimate or underestimate the true mutual information. Although this finite-sample estimator is not guaranteed in general to remain a lower bound exactly, in the experiments considered later it consistently underestimated the reference mutual information.

3.2 From Bound to Estimator

However, the bound (18) involves an expectation over the joint distribution $p(s, x)$ that we cannot compute directly. To turn it into something we can actually evaluate from data, we expand the expectation explicitly. Since S is discrete and X is continuous, as we saw in section 1, the expectation $\mathbb{E}[\log q(s|x)]$ is a sum over stimuli and an integral over responses:

$$\begin{aligned} \mathbb{E}[\log q(s|x)] &= \sum_{i=1}^K \int_{\mathbb{R}^d} p(s_i, x) \log q(s_i|x) dx && \text{(Definition of } \mathbb{E} \text{ for mixed variables)} \\ &= \sum_{i=1}^K \int_{\mathbb{R}^d} P(s_i) p(x|S=s_i) \log q(s_i|x) dx && \text{(Chain rule)} \\ &= \sum_{i=1}^K P(s_i) \int_{\mathbb{R}^d} p(x|S=s_i) \log q(s_i|x) dx && (P(s_i) \text{ does not depend on } x) \end{aligned} \quad (19)$$

We can now combine this with the entropy term $H(S) = -\sum_i P(s_i) \log P(s_i)$ from our bound (18). Bringing $-\log P(s_i)$ inside the inner integral and combining the logarithms gives

$$H(S) + \mathbb{E}[\log q(s|x)] = \sum_{i=1}^K P(s_i) \int_{\mathbb{R}^d} p(x|S=s_i) \log \frac{q(s_i|x)}{P(s_i)} dx \quad (20)$$

which is exactly the mutual information $I(S; X)$ (10) when $q(s|x) = p(s|x)$. Now, for each stimulus s_i the inner integral

$$\int_{\mathbb{R}^d} p(x|S=s_i) \log \frac{q(s_i|x)}{P(s_i)} dx = \mathbb{E}_{X|S=s_i} \left[\log \frac{q(s_i|X)}{P(s_i)} \right] \quad (21)$$

is an expectation under the conditional distribution of responses given stimulus s_i and for each stimulus level the experimental data $\{x_1^i, \dots, x_{n_i}^i\}$ is a sample from this distribution. Hence, using the law of large numbers [Jetka et al. \(2019\)](#), replace the integral with a sample average to get the following MI approximation

$$I(S; X) \approx \hat{I}(S; X) = \sum_{i=1}^K P(s_i) \frac{1}{n_i} \sum_{l=1}^{n_i} \log \frac{q(s_i | x_l^i)}{P(s_i)} \quad (22)$$

where n_i is the number of observed responses for stimulus s_i and $q(s_i | x_l^i)$ is the predicted probability of the classifier that the l -th response came from stimulus s_i . This is the basic empirical approximation used in the SLEMI algorithm introduced by [Jetka et al. \(2019\)](#). In their implementation, they use logistic regression to obtain the approximation $q(s|x)$ noting that "in principle, other classifiers could also be considered" without pursuing this further.

4 Decoding-Based Estimation

The probability-based approach of Section 3 relies on the classifier providing accurate estimates of the conditional probabilities $q(s | x)$. In practice, however, some classifiers are designed primarily for classification accuracy rather than for calibrated probability estimation. For such methods, it is more natural to use only the predicted label $\hat{s} = h(x)$ and not the full vector of estimated class probabilities. [Cepeda-Humerez et al. \(2019\)](#) propose a mutual-information estimator based on this idea: instead of using the conditional probabilities $q(s | x)$, the method uses the predicted labels and quantifies the information contained in the resulting pattern of correct and incorrect classifications.

4.1 The Confusion Matrix Approach

Once we have trained a classifier $h : \mathbb{R}^d \rightarrow \mathcal{S}$, we need to evaluate it on data it has not seen during training to avoid over-fitting. We take these unseen test responses, run them through the classifier, and compare each prediction to the true stimulus. The results are summarised in what we call a confusion matrix: a $K \times K$ table (one row and column for each of the K stimulus levels) where each entry is

$$\pi_{ij} = P(\hat{S} = s_j | S = s_i) \quad (23)$$

the fraction of responses from stimulus s_i that the classifier labels as s_j where $\hat{S} = h(X)$ is the classifier's prediction. Notice that each row sums to one since every response gets assigned to exactly one class. To build intuition, consider the two extreme cases:

$$\pi_{\text{perfect}} = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 1 \end{pmatrix} \quad \pi_{\text{useless}} = \begin{pmatrix} \frac{1}{K} & \frac{1}{K} & \cdots & \frac{1}{K} \\ \frac{1}{K} & \frac{1}{K} & \cdots & \frac{1}{K} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{1}{K} & \frac{1}{K} & \cdots & \frac{1}{K} \end{pmatrix} \quad (24)$$

A perfect classifier always predicts the correct stimulus, giving the identity matrix on the left. In contrast, a useless classifier guesses uniformly at random regardless of the true stimulus, giving identical rows on the right. In practice, real classifiers fall somewhere in between, and the pattern of off-diagonal entries tells us exactly which stimuli the classifier confuses with each other.

From the confusion matrix and the known stimulus distribution $P(S)$, we can recover the joint distribution of the true and predicted stimulus. By the chain rule of probability, the joint probability of the true stimulus s_i and the classifier predicting s_j is

$$P(S = s_i, \hat{S} = s_j) = P(S = s_i) \cdot P(\hat{S} = s_j | S = s_i) = P(s_i) \pi_{ij} \quad (25)$$

since π_{ij} is by definition the conditional probability of predicting s_j given that the true stimulus was s_i . Summing over the true stimuli gives the marginal distribution of the predictions, $P(\hat{S} = s_j) = \sum_{i=1}^K P(s_i) \pi_{ij}$. Now that we have recovered both the joint and the marginals, and since S and $\hat{S}$ are both discrete, we can compute their mutual information directly using the definition (9) of MI:

$$I(S; \hat{S}) = \sum_{i=1}^K \sum_{j=1}^K P(s_i) \pi_{ij} \log \frac{\pi_{ij}}{P(\hat{S} = s_j)} \quad (26)$$

This is the mutual information computed from the confusion matrix of decoded and true labels, corresponding to Eq. 26 of [Cepeda-Humerez et al. \(2019\)](#). Unlike the probability-based estimator (22), this does not require us to model continuous response distributions explicitly or evaluate the corresponding integrals. Moreover, the classifier does not need to output probabilities. Instead, it simply needs predictions on enough data to fill in the confusion matrix.

The role of the classifier in this approach is not simply to produce accurate predictions, but to define a decoded variable $\hat{S}$ such that the joint distribution of $(S, \hat{S})$ preserves as much as possible of the dependence between S and X . Equivalently, the aim is to make $I(S; \hat{S})$ as close as possible to $I(S; X)$. Perfect prediction is one extreme in which this dependence is preserved as strongly as possible, while a useless classifier is the opposite extreme.

4.2 The Data Processing Inequality

So far we have computed $I(S; \hat{S})$, the mutual information between the true stimulus and the classifier’s prediction. But what we actually want is $I(S; X)$, the mutual information between the stimulus and the full response.

Consider the following chain of events: the stimulus S induces the response X through the biological channel, and then the classifier maps X to its prediction $\hat{S} = h(X)$. Importantly, since $\hat{S}$ is computed entirely from X , it has no direct access to S . Consequently, we can model these three variables as what is known as a Markov Chain

$$S \rightarrow X \rightarrow \hat{S} \tag{27}$$

meaning that once X is known, the prediction $\hat{S}$ no longer depends directly on S . This lets us invoke the data processing inequality ([Cover and Thomas, 1991](#), Theorem 2.8.1) which states that for any such chain

$$I(S; \hat{S}) \leq I(S; X) \tag{28}$$

meaning that the mutual information $I(S; \hat{S})$ between the true stimulus and the classifier’s prediction is less than the true mutual information $I(S; X)$. This is because processing can only destroy information, never create it. Hence, $I(S; \hat{S})$ is always a lower bound on $I(S; X)$ and while a better classifier will give a tighter bound, no classifier can ever exceed the true mutual information.

5 Neural Variational Estimation: MINE

The classifier-based approaches we covered in Sections 3 and 4 both estimate mutual information indirectly. As we have seen, they train a classifier to predict S from X , then extract an MI estimate from either its probabilities or its predictions. MINE (Mutual Information Neural Estimation), introduced by (Belghazi et al., 2018), use a different strategy. There is no classification step, no confusion matrix, and no need to estimate conditional probabilities $p(s|x)$ at all. Instead, MINE trains a neural network to directly estimate MI by solving an optimisation problem.

To apply MINE in our setting, we represent the discrete stimulus S in vector form. Previously, we treated S as a categorical random variable taking values in the finite set $\{s_1, \dots, s_K\}$. However, for neural estimators such as MINE, it is more convenient to encode each stimulus as a one-hot vector in $\mathbb{R}^K$. Thus, if the presented stimulus is category k we represent it by the standard basis vector $e_k \in \mathbb{R}^K$, whose k -th entry is 1 and all other entries are 0. With this encoding, S is now viewed as a K -dimensional random vector, while the response remains a trajectory $X \in \mathbb{R}^T$.

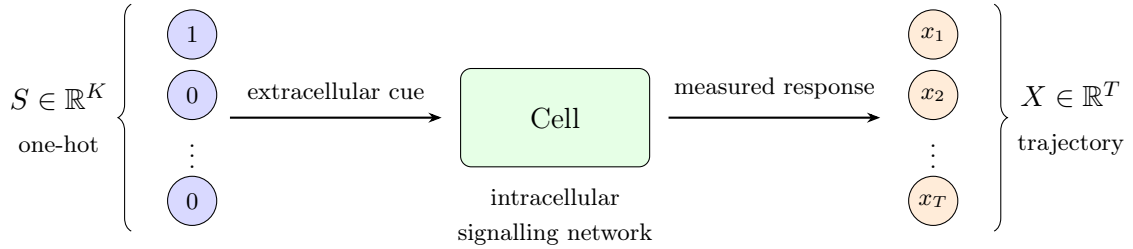


Figure 3: The signalling model with one-hot encoded stimulus.

To understand where this optimisation problem comes from, we need to look at a classical result about the KL divergence.

5.1 The Donsker-Varadhan Representation

Recall from Section 2 that mutual information is the KL divergence between the joint distribution and the product of the marginals:

$$I(S; X) = D_{\text{KL}}(p(s, x) \| p(s)p(x)).$$

Using the expectation form of KL divergence from (8), this can be written as

$$I(S; X) = \mathbb{E}_{p(s, x)} \left[\log \frac{p(s, x)}{p(s)p(x)} \right],$$

which depends on the density ratio $\frac{p(s, x)}{p(s)p(x)}$. In practice, this ratio is not known explicitly, so the mutual information cannot be evaluated directly from this expression.

Belghazi et al. (2018) avoid this difficulty by using a variational representation of KL divergence. Rather than estimating the density ratio directly, they optimise over a class of real-valued functions T . Specifically, the KL divergence admits the following dual representation (Donsker and Varadhan, 1983):

$$D_{\text{KL}}(P \| Q) = \sup_{T: \Omega \rightarrow \mathbb{R}} \left\{ \mathbb{E}_P[T] - \log \mathbb{E}_Q[e^T] \right\}. \quad (29)$$

where the supremum is taken over all functions T for which both expectations are finite. To interpret this expression, consider the two terms separately. The first term, $\mathbb{E}_P[T]$, is large when T takes large values on points where P places substantial mass. The second term, $\log \mathbb{E}_Q[e^T]$, penalises functions that are large under Q . Hence, the objective is maximised by functions that are large in regions where P is comparatively large relative to Q , and smaller where Q dominates. The optimal choice is

$$T^* = \log \frac{dP}{dQ} + C$$

for some constant C .

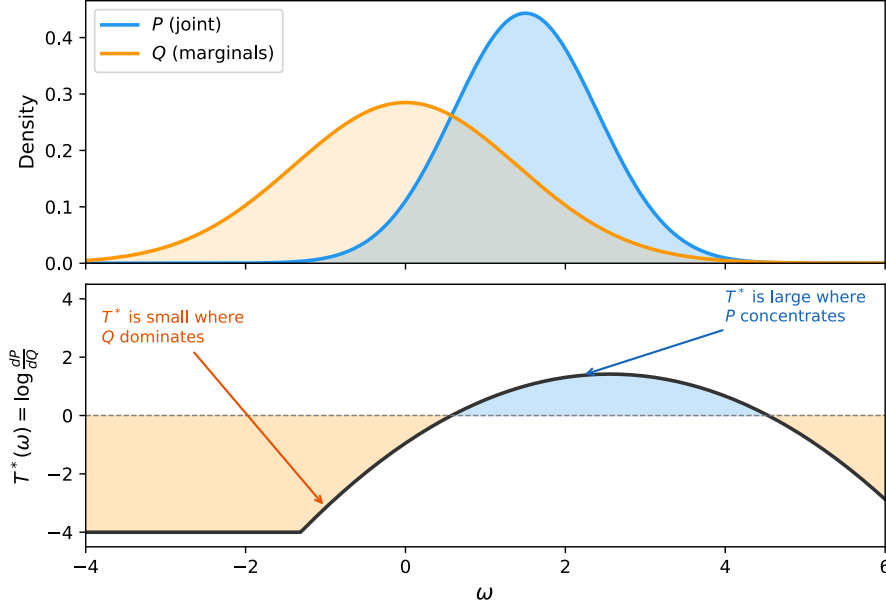


Figure 4: One-dimensional illustration of the DV representation. The optimal function $T^* = \log \frac{dP}{dQ}$ is large where the joint P concentrates relative to the marginals Q , and small where Q dominates.

Applying this to mutual information with $P = p(s, x)$ and $Q = p(s)p(x)$, we obtain

$$I(S; X) = \sup_{T: \mathbb{R}^K \times \mathbb{R}^T \rightarrow \mathbb{R}} \left\{ \mathbb{E}_{p(s, x)}[T(s, x)] - \log \mathbb{E}_{p(s)p(x)}[e^{T(s, x)}] \right\}. \quad (30)$$

Here, the first expectation is taken over stimulus-response pairs drawn from the joint distribution $p(s, x)$, whereas the second is taken over pairs drawn independently from $p(s)p(x)$. The difference between these two terms therefore quantifies how different the joint distribution is from the product of its marginals.

In practice, we cannot optimise over all admissible functions T . Instead, we restrict T to belong to a smaller function class $\mathcal{F}$. Since the supremum over a subset can never exceed the supremum over the full set, this gives the lower bound

$$I(S; X) \geq \sup_{T \in \mathcal{F}} \left\{ \mathbb{E}_{p(s, x)}[T(s, x)] - \log \mathbb{E}_{p(s)p(x)}[e^{T(s, x)}] \right\}. \quad (31)$$

This bound is tight when $\mathcal{F}$ is rich enough to contain the optimal function $T^* = \log \frac{p(s, x)}{p(s)p(x)} + C$ for some constant C (Belghazi et al., 2018). Mutual-information estimation is therefore reduced to the problem of finding, within a chosen function class, the function T that maximises the right-hand side of (31).

5.2 The MINE Estimator

The idea behind MINE is to approximate the function T in the variational bound (31) by a neural network. More precisely, we choose a neural-network architecture that defines a family of scalar-valued functions

$$\{T_\theta : \mathbb{R}^K \times \mathbb{R}^T \rightarrow \mathbb{R}\}_{\theta \in \Theta},$$

where θ denotes the collection of all trainable parameters, namely the weights and biases of the network. For each fixed value of θ , the network computes a function $T_\theta(s, x)$, and MINE seeks the parameter values that maximise the variational objective in (31).

A neural network is a parametric function built from successive affine transformations and nonlinear activation functions. Given sufficient width and depth, such networks can approximate a large class of continuous mappings (Hornik et al., 1989). The parameters θ are learned by gradient ascent: starting from an initial value, we repeatedly update θ in the direction that increases the objective.

In practice, we do not have access to the true distributions $p(s, x)$ and $p(s)p(x)$, so we replace the expectations in the bound (31) with sample averages. Given a mini-batch of b pairs $\{(s_i, x_i)\}_{i=1}^b$ drawn from the data, we can estimate the first expectation directly since these are samples from the joint distribution. However, for the second expectation, we need pairs where the dependence between s and x has been broken down. To get these Belghazi et al. (2018) take the same minibatch and randomly shuffle the stimulus labels: let $\bar{s}_i = s_{\pi(i)}$ where π is a random permutation. Now each response x_i is paired with a stimulus that did not actually produce it, breaking the dependence between them. This gives us the empirical objective

$$V(\theta) = \frac{1}{b} \sum_{i=1}^b T_\theta(s_i, x_i) - \log \left(\frac{1}{b} \sum_{i=1}^b e^{T_\theta(\bar{s}_i, x_i)} \right) \quad (32)$$

which we maximise over θ by gradient ascent. At convergence, $V(\theta)$ is the MINE estimate of $I(S; X)$.

However, when we compute the gradient of the objective (32) using a minibatch, the second term cause trouble. Indeed, the derivative of $\log(f)$ is $1/f$, so differentiating the log in the second term puts $\frac{1}{b} \sum_i e^{T_\theta(\bar{s}_i, x_i)}$ in the denominator of the gradient. But we are estimating this denominator from a single minibatch, and dividing by a noisy estimate introduces a systematic bias that does not vanish with more training steps. Belghazi et al. (2018) fix this by maintaining a running average m of the denominator that is updated at each step, giving a much more stable value to divide by. For small learning rates, this makes the bias arbitrarily small. The full procedure is summarised in Algorithm 1 below.

Algorithm 1 MINE Training Procedure

Require: Training data $\{(s_i, x_i)\}_{i=1}^n$, network T_θ , learning rate α , EMA decay γ

- 1: Initialise network parameters θ
 - 2: Initialise moving average $m \leftarrow 1$
 - 3: **for** epoch = 1, ..., E **do**
 - 4: Draw minibatch $\{(s_i, x_i)\}_{i=1}^b$ from data
 - 5: Shuffle stimulus labels: $\bar{s}_i \leftarrow s_{\pi(i)}$
 - 6: $V(\theta) \leftarrow \frac{1}{b} \sum_{i=1}^b T_\theta(s_i, x_i) - \log\left(\frac{1}{b} \sum_{i=1}^b e^{T_\theta(\bar{s}_i, x_i)}\right)$
 - 7: Update $m \leftarrow (1 - \gamma) \cdot m + \gamma \cdot \frac{1}{b} \sum_{i=1}^b e^{T_\theta(\bar{s}_i, x_i)}$
 - 8: Compute corrected gradient using m
 - 9: $\theta \leftarrow \theta + \alpha \cdot \nabla_\theta V(\theta)$
 - 10: **end for**
 - 11: **return** $V(\theta)$ as MI estimate
-

More precisely, in MINE the function T in the variational bound is parametrised by a feed-forward neural network. For each choice of parameters θ (the network weights and biases), the network defines a scalar-valued function $T_\theta(s, x)$. The input is the concatenation of the encoded stimulus s and response trajectory x , and the output is a single real number. Figure 5 illustrates the general structure used in this project.

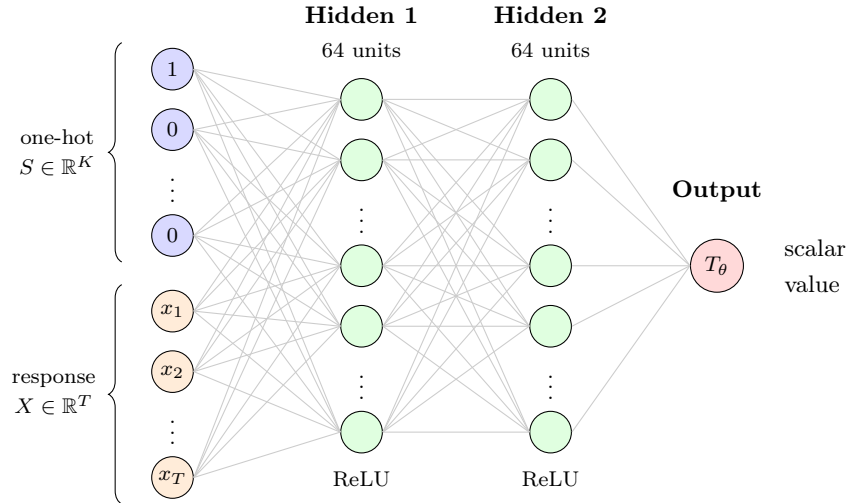


Figure 5: Schematic architecture used to parametrise the function $T_\theta(s, x)$ in MINE. The input is the concatenation of the encoded stimulus s and the response trajectory x , and the network outputs a single scalar value.

The precise network size and training hyperparameters are implementation choices rather than part of the definition of MINE. The specific architecture and optimisation settings used in the experiments are described later in Section 7.1.

6 Vector Copula Estimation (VCE)

In Section 5, we presented MINE as a direct neural approach to mutual information estimation. MINE uses a single neural network to approximate the density ratio $p(s, x)/p(s)p(x)$ directly from data, by optimising the Donsker-Varadhan bound via gradient ascent. While this approach is theoretically appealing (the DV bound is tight and MINE is strongly consistent) it faces a practical difficulty that motivates the alternative approach we describe in this section.

The first is the curse of high discrepancy (Srivastava et al., 2023). MINE works by contrasting samples from the joint distribution $p(s, x)$ against samples from the product of marginals $p(s)p(x)$. When mutual information is high, these two distributions look very different from each other, and telling them apart becomes a hard task for the neural network. In our cell-signalling setting, where mutual information is expected to be modest, on the order of less than 1 or 2 bits, this issue is unlikely to be severe. Nevertheless, MINE still asks its neural network to untangle two things at once: the individual structure of each variable and the dependence between them. Even at low MI, this conflation makes the estimator less efficient than it needs to be, because much of the neural network’s capacity is spent learning the marginal features that are irrelevant to the quantity we actually want to estimate.

This raises the question of whether we can separate these two tasks. If we could strip away the marginal structure of S and X , making each variable individually featureless, then a subsequent estimator would only need to detect whatever statistical relationship remains between them. This is the idea behind the Vector Copula Estimator of Chen et al. (2025). Rather than asking one network to handle everything, VCE splits the problem in two: it first learns the marginal distributions so that their effect can be removed, and then estimates the dependence that remains. Mutual information depends only on this dependence structure, not on the marginals themselves. The marginals are learned only as an intermediate step needed to transform the data into a representation where the dependence can be estimated directly.

6.1 Vector Copula Theory

The classical Sklar theorem (Nelsen, 2006) states that any joint distribution can be decomposed into its marginals and a copula, which captures the dependence structure between the variables. The recent vector copula theory of Fan and Henry (2023) generalises this result from univariate to multivariate marginals, which is the setting we need for our signalling problem, where both the encoded stimulus and the response are represented as vectors.

The key object in this theory is the vector rank. Given a multivariate distribution p on $\mathbb{R}^d$, the vector rank is a map $u = \nabla\psi^{-1}$ that transforms p into the uniform distribution on $[0, 1]^d$, where ψ is a convex function. A uniform distribution is one in which every region of the same volume is equally likely. Thus, the vector rank removes the marginal shape of the distribution and maps it to a standard reference form. When $d = 1$, this reduces to the usual probability integral transform: if X has cumulative distribution function F , then $F(X)$ is uniformly distributed on $[0, 1]$. In higher dimensions, however, applying this construction coordinate by coordinate would ignore the dependence between dimensions. The vector rank instead acts jointly on all coordinates, so that the result is uniform over the full d -dimensional space $[0, 1]^d$.

Now that we have described vector ranks, we can state the main result. The vector Sklar theorem

of [Fan and Henry \(2023\)](#) shows that any joint distribution can be decomposed into its marginal distributions and a vector copula C that captures the dependence between them, generalising Sklar’s theorem from univariate to multivariate marginals. In density form, this gives

$$p(x, y) = p(x) p(y) c(u_X, u_Y), \quad (33)$$

where u_X and u_Y are the vector ranks of the two variables, and $c(u_X, u_Y)$ is the vector copula density, the derivative of the copula C . This splits the joint density into three parts: $p(x)$ describes the first variable on its own, $p(y)$ describes the second on its own, and $c(u_X, u_Y)$ captures everything about how they relate to each other.

The connection to mutual information then follows in the continuous multivariate setting considered by [Chen et al. \(2025\)](#). Under the assumptions of that theory, which is that the marginals are absolutely continuous with support in convex sets so that the vector-rank maps exist and are almost-everywhere invertible, mutual information is preserved under the transformation from (X, Y) to (u_X, u_Y) . Using the entropy form of mutual information ([Cover and Thomas, 1991](#)), this yields

$$I(X; Y) = I(u_X; u_Y) = H(u_X) + H(u_Y) - H(u_X, u_Y).$$

Because vector ranks are uniform by construction, their individual differential entropies are zero, so

$$I(X; Y) = -H[c(u_X, u_Y)], \quad (34)$$

where $H[\cdot]$ is the differential entropy as defined in Section 2. A full derivation of (34) is given in Appendix A.

Strictly speaking, however, this identity is proved in [Chen et al. \(2025\)](#) for continuous multivariate variables. In our signalling problem, the original stimulus S is discrete. Although one-hot encoding allows S to be processed by neural estimators, it does not make the stimulus marginal absolutely continuous. Therefore, we do not apply (34) as a theorem literally to the original mixed discrete-continuous pair (S, X) . Instead, we use it as theoretical motivation for a VCE-style estimator adapted to our setting. In other words, the core idea is to remove as much marginal structure as possible and then estimate the remaining dependence, while assessing empirically whether this strategy works well in our synthetic signalling experiments.

This perspective, extending earlier univariate work by [Ma and Sun \(2011\)](#), says something simple but powerful: mutual information is determined by the dependence structure captured by the copula density. Hence, if we can estimate that dependence structure well, we can estimate MI well, even when the marginal distributions themselves are complicated.

6.2 Learning The Marginal Learning with Flow Matching

Equation (34) tells us that to estimate mutual information, we need to estimate the copula density $c(u_S, u_X)$, which in turn requires computing the vector ranks u_S and u_X . But computing the true vector rank requires knowing the true marginal distribution exactly, which we do not. [Chen et al. \(2025\)](#) approximate the vector ranks in two step, as shown in Figure 6: first, learn each marginal distribution using a flow model, and then apply an element-wise ranking to the result. We describe the first step here and the second in Section 6.3.

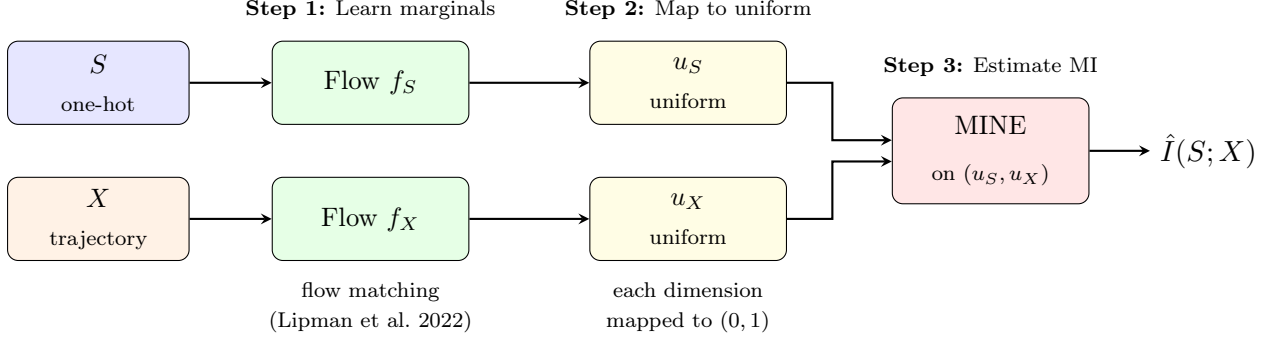


Figure 6: Pipeline of the VCE estimator (Algorithm 2, [Chen et al. 2025](#)).

The idea is to train a flow model f_S on stimulus data and a separate flow model f_X on response data, each learning its respective marginal distribution independently. Once trained, each flow provides an invertible map between the data distribution and a standard Gaussian $\mathcal{N}(0, I)$ where I is the identity matrix. These flows are trained using flow matching ([Lipman et al., 2023](#)), which we now describe.

Consider learning the marginal $p(x)$. Flow matching defines a straight-line path between a noise sample $x_0 \sim \mathcal{N}(0, I)$ and a data point x_1 from the dataset.

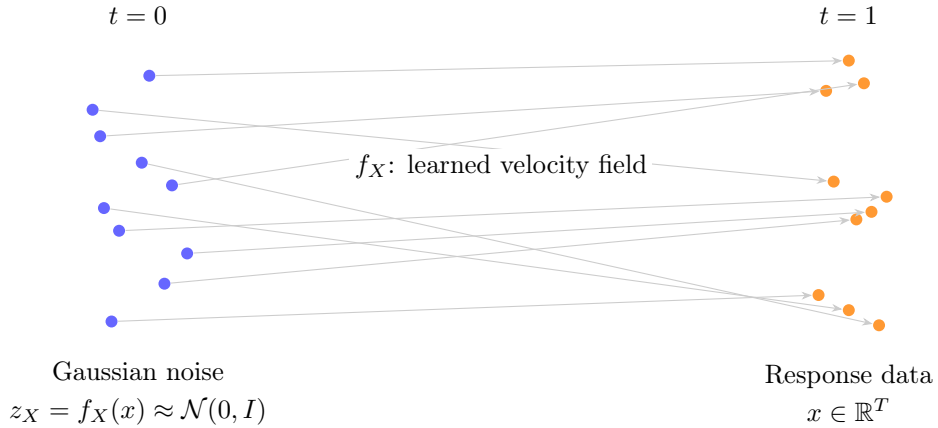


Figure 7: Flow matching training paths.

At any time $t \in [0, 1]$, a point along this path is

$$x_t = (1 - t) x_0 + t x_1 \quad (35)$$

Since the path is a straight line, the velocity (derivative dx/dt) at any point is constant and simply equal to $x_1 - x_0$. A neural network $v_\theta(t, x_t)$ is then trained to predict this velocity by

minimising

$$\mathcal{L}_{\text{FM}} = \mathbb{E}_{t, x_0, x_1} \left[\|v_\theta(t, x_t) - (x_1 - x_0)\|^2 \right], \quad (36)$$

where t is sampled uniformly from $[0, 1]$, x_0 from $\mathcal{N}(0, I)$, and x_1 from the response data X . At each training step, we sample a random noise point x_0 , a random data point x_1 , and a random time t . We compute the interpolated point x_t using Equation (35), and then feed only x_t and t into the network. The network never sees x_0 or x_1 directly, it has to figure out the right velocity dx/dt , from where it is, x_t , and when, t . Since many different straight-line paths pass through roughly the same region of space at the same time, the network learns the average velocity across all of them. Over many training steps, this averaging produces a smooth velocity field that, as a whole, transports the Gaussian distribution $\mathcal{N}(0, I)$ into the data distribution X .

Once trained, this velocity field defines an ordinary differential equation: $dx/dt = v_\theta(t, x)$. Integrating this ODE from $t = 0$ to $t = 1$ maps noise to data; while integrating from $t = 1$ to $t = 0$ maps data to noise. It is the reverse direction that we need, so given a response x , we integrate backwards to obtain a noise-space representation $z_X = f_X(x)$, which is approximately standard Gaussian. The same procedure is applied separately to the stimulus, giving $z_S = f_S(s)$.

6.3 Computing Vector Ranks

After the flow step, we have noise-space representations $z_s \approx \mathcal{N}(0, I)$ and $z_X \approx \mathcal{N}(0, I)$ of the stimulus S and the response X respectively. But the VCE requires variables that are uniform on $[0, 1]^K$ and $[0, 1]^T$, not Gaussian. So a second step is needed: we apply an element-wise ranking that maps each dimension from approximately Gaussian to approximately uniform (Chen et al., 2025).

For each dimension j independently, we replace every value with its rank among all n samples, divided by $n + 1$

$$\hat{u}_{ij} = \frac{\text{rank}(z_{ij})}{n + 1}, \quad (37)$$

where z_{ij} is the j -th dimension of the i -th sample in noise space. This produces values in $(0, 1)$ that are empirically approximately uniform in each dimension.

Although ranking each dimension makes it uniform, this alone does not recover the true vector rank for multivariate data. In the multivariate setting, vector rank is defined as a joint transport map from the full distribution to a reference distribution, rather than as a collection of separate univariate rank transforms (Chernozhukov et al., 2017). So for our response trajectories in $\mathbb{R}^T$, ranking each time step independently only makes each time step marginally uniform, it does not account for the correlations between time steps. For AR(1) trajectories, where adjacent time steps are strongly dependent, this is a poor approximation.

The flow fixes this by first mapping the full trajectory distribution to approximately $\mathcal{N}(0, I)$. In that space, the dimensions are approximately independent, so the true vector rank map breaks down into separate per-dimension maps. Ranking each dimension separately is then justified, and gives an estimate of the true vector rank (Chen et al., 2025).

6.4 Estimating the Copula Density

With the vector ranks $\hat{u}_S$ and $\hat{u}_X$ computed, the final step is to estimate the copula density $c(\hat{u}_S, \hat{u}_X)$, from which we obtain the MI estimate via Equation (34). [Chen et al. \(2025\)](#) propose two approaches. VCE1 (Algorithm 1) models the copula as a mixture of parametric Gaussian copulas, which assumes a specific form for the dependence structure. VCE2 (Algorithm 2) estimates the copula density using a neural network, making no explicit assumption about its form. We use VCE2 because the relationship between a discrete stimulus S and continuous AR(1) trajectories X with class-dependent dynamics may have a complex, non-Gaussian dependence structure that a mixture of Gaussian copulas would not capture well.

VCE2 works as follows. We first choose a reference copula c' , a distribution over the rank space $[0, 1]^{K+T}$ that we can easily sample from. We then draw n synthetic rank pairs $(\tilde{u}_S, \tilde{u}_X)$ from c' . Now we have two sets of rank pairs: the real ones $(\hat{u}_S, \hat{u}_X)$ estimated from our data in Section 6.3, and the synthetic ones from c' . We train a classifier r to tell the two sets apart. At optimum, this classifier learns the density ratio between the true copula and the reference ([Gutmann and Hyvärinen, 2010](#)):

$$r(\hat{u}_S, \hat{u}_X) \approx \frac{c(\hat{u}_S, \hat{u}_X)}{c'(\hat{u}_S, \hat{u}_X)}. \quad (38)$$

The copula density is then recovered as $\hat{c} = r \cdot c'$, and the MI estimate is

$$\hat{I}(S; X) = \frac{1}{n} \sum_{i=1}^n \log \hat{c}(\hat{u}_S^{(i)}, \hat{u}_X^{(i)}) = \frac{1}{n} \sum_{i=1}^n \log \left[r(\hat{u}_S^{(i)}, \hat{u}_X^{(i)}) \cdot c'(\hat{u}_S^{(i)}, \hat{u}_X^{(i)}) \right]. \quad (39)$$

Following [Chen et al. \(2025\)](#), we set $c' = 1$, the independent copula. This is the copula density of two variables with no dependence at all, it is constant and equal to 1 everywhere on $[0, 1]^{K+T}$. Sampling from it simply means drawing each dimension independently from Uniform(0,1). With this choice, the density ratio becomes $r = c/1 = c$, so the classifier directly estimates the copula density itself. We also follow [Chen et al. \(2025\)](#) in using the MINE loss (the Donsker-Varadhan objective from Section 5) for this density ratio estimation. The resulting objective is

$$\hat{I}(S; X) = \sup_{T_\theta} \left\{ \frac{1}{n} \sum_{i=1}^n T_\theta(\hat{u}_S^{(i)}, \hat{u}_X^{(i)}) - \log \frac{1}{n} \sum_{i=1}^n e^{T_\theta(\tilde{u}_S^{(i)}, \tilde{u}_X^{(i)})} \right\} \quad (40)$$

where $\tilde{u}_S$ denotes shuffled stimulus ranks. This is identical to standard MINE we described in the previous section, but applied to the vector ranks rather than the original data. The difference is that the neural network now operates in a space where the marginal complexity has already been removed. The flows and ranking have made each variable individually uniform, so any pattern the neural network detects must come from the dependence between S and X , not from the individual structure of either the stimulus or the response.

The full VCE2 estimator, combining the marginal learning, vector rank computation, and copula estimation steps described in the previous subsections, is summarised in Algorithm 2 below.

Algorithm 2 VCE Estimator (Algorithm 2, Chen et al. 2025)

Require: Training data $\{(s_i, x_i)\}_{i=1}^n$, flow models f_S, f_X , MINE neural network T_θ

```
1: // Step 1: Learn marginals via flow matching
2: Train  $f_S$  on stimulus data  $\{s_i\}_{i=1}^n$  using  $\mathcal{L}_{\text{FM}}$  (Equation 36)
3: Train  $f_X$  on response data  $\{x_i\}_{i=1}^n$  using  $\mathcal{L}_{\text{FM}}$  (Equation 36)
4: // Step 2: Compute vector ranks
5: for  $i = 1, \dots, n$  do
6:    $z_S^{(i)} \leftarrow f_S(s_i)$  {ODE integration  $t = 1 \rightarrow 0$ }
7:    $z_X^{(i)} \leftarrow f_X(x_i)$  {ODE integration  $t = 1 \rightarrow 0$ }
8: end for
9: for each dimension  $j$  do
10:   $\hat{u}_{ij} \leftarrow \text{rank}(z_{ij}) / (n + 1)$  {element-wise ranking}
11: end for
12: // Step 3: Estimate MI via MINE in rank space
13: Run MINE (Algorithm 1) on  $\{(\hat{u}_S^{(i)}, \hat{u}_X^{(i)})\}_{i=1}^n$ 
14: return  $\hat{I}(S; X)$ 
```

In summary, VCE2 does not avoid learning the marginals. Instead, it learns them first with separate flow models, and then estimates the dependence in the resulting rank space. Its advantage over standard MINE is therefore not that it spends no capacity on the marginals, but that it does not try to learn the marginals and the dependence at the same time in one network: it is a divide and conquer approach. The final neural step is carried out on variables that are approximately marginally uniform, so it can focus more directly on the remaining dependence between S and X . In the following section, we compare VCE2 against MINE and the classifier-based estimators from Sections 3 and 4 on our synthetic AR(1) data.

7 Experimental Comparison

In this section, we compare all four estimation methods on the synthetic AR(1) signalling data introduced in Section 1. The goal is to understand how each estimator behaves as we vary two key features of the problem: the separation between stimulus classes, which controls how much mutual information is available, and the trajectory length T , which controls the dimensionality of the response.

7.1 Simulation Setup

The synthetic data follow the class-dependent AR(1) model defined in Equation (1), with both Gaussian and Student- t ($\nu = 3$) innovations. To set up the comparison, we need to decide which stimulus classes to use, how to vary the true mutual information across a wide range, and which classifiers to pair with the probability-based and decoding-based estimators.

Stimulus classes. I used $K = 8$ stimulus classes, each with a different combination of persistence α_k , baseline mean μ_k , and noise level τ_k , as listed in Table 1. This number of classes is representative of the range considered in the biological literature. For example, [Granados et al. \(2018\)](#) use between four to seven environmental states in yeast, while [Tang and Hoffmann \(2022\)](#) compare estimation methods using NF κ B response to 13 immune stimuli. With eight classes the entropy ceiling is $H(S) = \ln 8 \approx 2.08$ nats, which gives enough room for the mutual information to grow with T before saturating, while remaining in the moderate-information regime that is characteristic of intracellular signalling. The parameters were chosen to create a range of different stimulus classes. Some classes have slow decay and low noise, others have fast decay and high noise, and their means are spread across distinct levels.

Table 1: AR(1) parameters for each of the $K = 8$ stimulus classes. The Student- t generator uses the same μ_k , α_k , τ_k values with $\nu = 3$ degrees of freedom.

Class k	μ_k	α_k	τ_k
0	0.5	0.8	0.4
1	1.5	0.3	0.5
2	2.5	0.6	0.3
3	3.5	0.4	0.6
4	1.0	0.7	0.5
5	2.0	0.2	0.4
6	3.0	0.5	0.7
7	0.8	0.9	0.3

To give a clearer picture of the simulated data, Figures 8 and 9 show example trajectories from the Gaussian and Student- t AR(1) generators respectively, using the class parameters in Table 1 with $\lambda = 1$. Here λ is the global mean-separation scale, which allows the true mutual information to be varied systematically, and is discussed in more detail below.

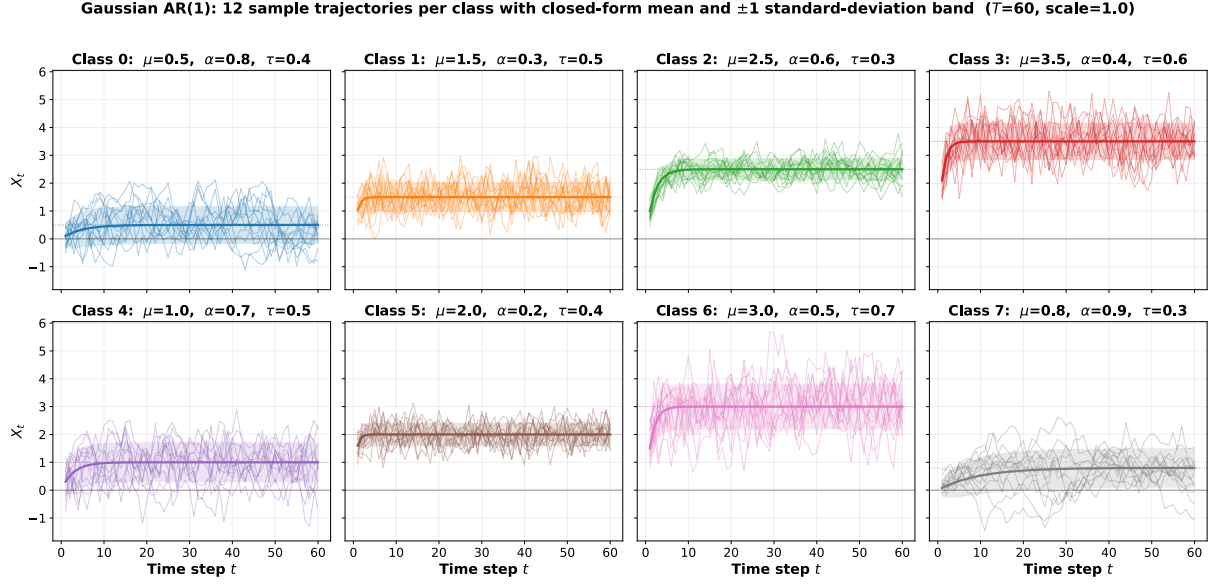


Figure 8: Sample trajectories from the Gaussian AR(1) generator for each of the $K = 8$ stimulus classes (Table 1) at $\lambda = 1$.

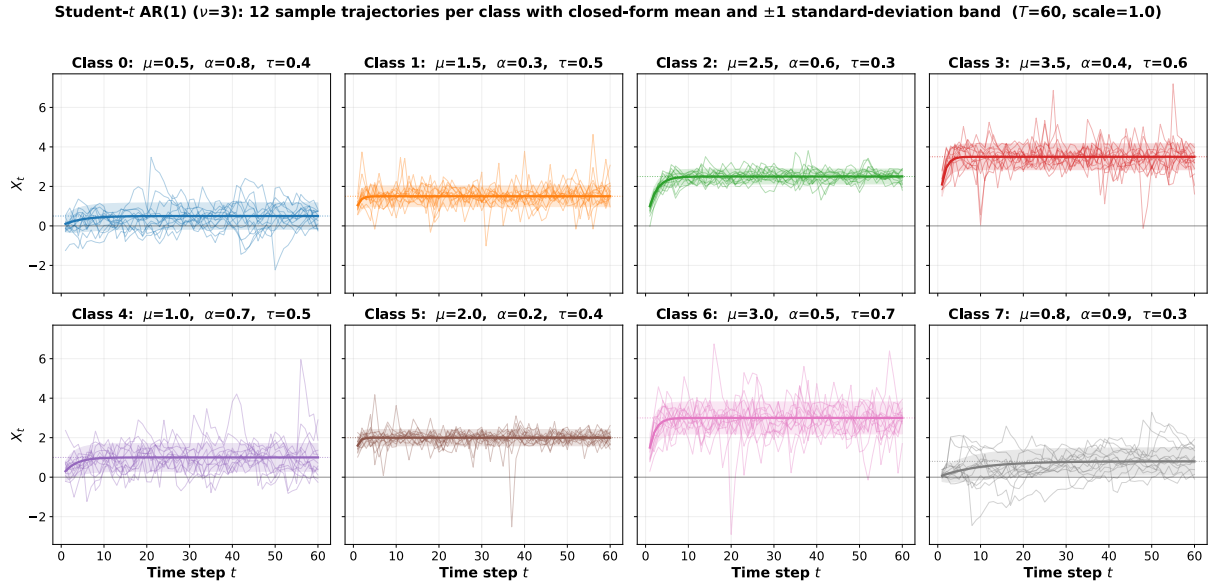


Figure 9: Sample trajectories from the Student- t AR(1) generator ($\nu = 3$) for each of the $K = 8$ stimulus classes at $\lambda = 1$.

Controlling the true mutual information. To vary the true mutual information $I(S; X)$ in a controlled way, I introduced a global mean-separation scale $\lambda > 0$ that multiplies all class means simultaneously. That is, the AR(1) parameters for class k at scale λ are $(\lambda\mu_k, \alpha_k, \tau_k)$, with all other parameters unchanged. When λ is small, the class means are pushed close together and the response distributions overlap heavily, so the mutual information is low. As λ increases, the classes separate and the mutual information grows toward its ceiling $H(S)$. This design lets us sweep the true MI from near zero to near saturation using a single parameter, while keeping the overall structure of each class the same. In the experiments below, λ takes values in $\{0.05, 0.1, 0.2, 0.5, 1.0, 2.0, 3.0\}$. Additionally, the trajectory length T was varied over

$\{5, 10, 20, 40, 80, 120\}$, covering low-dimensional settings ($T \leq 20$) where the problem is modest, moderate dimensionality ($T = 40$ to 80), and a biologically realistic range ($T = 120$) comparable to what is seen in live-cell imaging experiments. For example, when [Cepeda-Humerez et al. \(2019\)](#) re-analyses the yeast data of [Granados et al. \(2018\)](#), sampled every 2.5 minutes, the trajectories have $d = 45$ time points for the main analysis and up to $d = 170$ in extended tests, while the mammalian Ca^{2+} signalling data they consider reach $d = 200$.

Ground-truth mutual information. The reference value of $I(S; X)$ used to evaluate the performance of each estimator is computed by Monte Carlo approximation ([Cepeda-Humerez et al., 2019](#)) of the identity

$$I(S; X) = \mathbb{E}[\log p(X | S) - \log p(X)], \quad (41)$$

where $p(X) = \sum_{k=1}^K P(s_k) p(X | S = s_k)$ is the overall response density, obtained by averaging the class-conditional densities over the stimulus distribution ([Cover and Thomas, 1991](#)). To evaluate (41), I drew $N = 200,000$ samples $(s^{(r)}, x^{(r)})$ from the synthetic AR(1) model and computed

$$I(S; X) \approx \frac{1}{N} \sum_{r=1}^N \left[\log p(x^{(r)} | s^{(r)}) - \log \sum_{j=1}^K P(s_j) p(x^{(r)} | S = s_j) \right]. \quad (42)$$

This is possible because we specified the AR(1) model ourselves, so the class-conditional densities $p(x | S = s_k)$ are available in closed form for both generators. For the Gaussian AR(1), each class-conditional trajectory distribution is multivariate Gaussian with a mean vector and covariance matrix derived from the AR(1) recurrence, so the log-density can be evaluated directly. For the Student- t AR(1), the process is first-order Markov, so the class-conditional density factorises as

$$p(x_{1:T} | S = s_k) = p(x_1 | S = s_k) \prod_{t=2}^T p(x_t | x_{t-1}, S = s_k), \quad (43)$$

where each conditional $p(x_t | x_{t-1}, S = s_k)$ is a scaled Student- t density whose parameters are set by the AR(1) recurrence. Hence, the log-densities were evaluated using these factorisations throughout. With $N = 200,000$, the Monte Carlo standard error is negligible relative to the differences between the estimation methods, so the resulting values can be treated as exact for comparison purposes.

Classifiers. The classifier-based estimators from Sections 3 and 4 require a trained classifier to produce either predicted probabilities or predicted labels. I used three classifiers that represent different modelling assumptions:

- **Quadratic Discriminant Analysis (QDA)** ([Hastie et al., 2009](#)), which fits a separate Gaussian distribution to each class. This is a good choice when the data are Gaussian, and its assumptions match the Gaussian AR(1) generator, but because these parameters must be estimated from data, QDA can still suffer from estimation error when the sample size is small relative to the dimension.
- **Support Vector Machine with RBF kernel and Platt scaling (SVM)** ([Cortes and Vapnik, 1995](#)), which uses a radial basis function kernel to find the boundary between

classes that separates them with the widest possible margin. On its own, an SVM only outputs a class label, so Platt scaling (Platt, 1999) is applied afterwards to convert these labels into probability estimates. Unlike QDA, the SVM makes no assumption about the shape of each class’s distribution.

- **Random Forest (RF)** (Breiman, 2001), which builds a large collection of decision trees, each of which splits the data along different features and thresholds, and then averages their predictions. Random forests are widely used in practice but are known to produce probability estimates that tend toward $1/K$.

Each classifier was evaluated using stratified 5-fold cross-validation. That is, the data were split into five parts, the classifier was trained on four parts and tested on the remaining part, and this was repeated until every trajectory x had been predicted once by a model that had not seen it during training. The held-out outputs were then used in two ways: the probability-based estimator (Section 3) used the predicted class probabilities, while the decoding-based estimator (Section 4) used only the predicted labels.

Neural estimator hyperparameters. For MINE, the function T_θ was parametrised by a feed-forward neural network with two hidden layers of 64 units each and ReLU activations. The input was the concatenation of the one-hot encoded stimulus and the response trajectory, giving input dimension $K + T$. Before training, each coordinate of the response trajectory was standardised to have zero mean and unit variance across the sample. This preprocessing does not change the mutual information, since it is an invertible transformation of the data, but it improves numerical conditioning during optimisation. The network was trained with Adam using a learning rate of 10^{-3} , minibatch size $b = 512$, and 3,000 iterations. To reduce the gradient bias in the Donsker-Varadhan objective, an exponential moving average correction was used with decay parameter $\gamma = 0.01$. For numerical stability, the maximum value of T_θ on the marginal samples in each minibatch was subtracted before exponentiation. The final MINE estimate was obtained by averaging the objective values over the last 20% of training iterations.

For VCE, the implementation follows Section 6. The flow-matching models used to learn the marginals each had four hidden layers of 512 units with Softplus activation,

$$\text{Softplus}(z) = \log(1 + e^z),$$

which is smoother than ReLU and therefore better suited to learning velocity fields. Each flow was trained for 200 epochs, with the learning rate starting at 5×10^{-4} and decreasing gradually to zero over training. The learned velocity field was then integrated backwards in time using an adaptive ODE solver with tolerances of 10^{-5} in order to map data to noise space, after which element-wise ranking produced the vector ranks. Mutual information was then estimated by running MINE on these ranks, using a neural network with two hidden layers of 128 units and the same Donsker-Varadhan objective and exponential-moving-average correction as in the MINE estimator. The overall implementation follows Algorithm 2 of Chen et al. (2025), but the network sizes were reduced to suit the smaller scale of the present problem: the flow hidden dimension was halved from 1024 to 512, the rank-space network was reduced from three hidden layers of 500 units to two hidden layers of 128 units, and early stopping was replaced by a fixed 200 epochs so that all $n = 5,000$ samples could be used for marginal learning.

7.2 Results

Figures 10 and 12 show the signed bias (estimated MI minus true MI) for the six classifier-based estimator variants, and Figures 13 and 14 show the same for MINE and VCE. Blue cells mark underestimation and red cells mark overestimation. Across all four figures, the main trend is that estimators consistently underestimate MI. The worst negative biases tend to appear when the mean-separation scale λ is small and the trajectory length T is large. This weak-signal, high-dimensional corner of the grid turns out to be the hardest regime for nearly every method I tested.

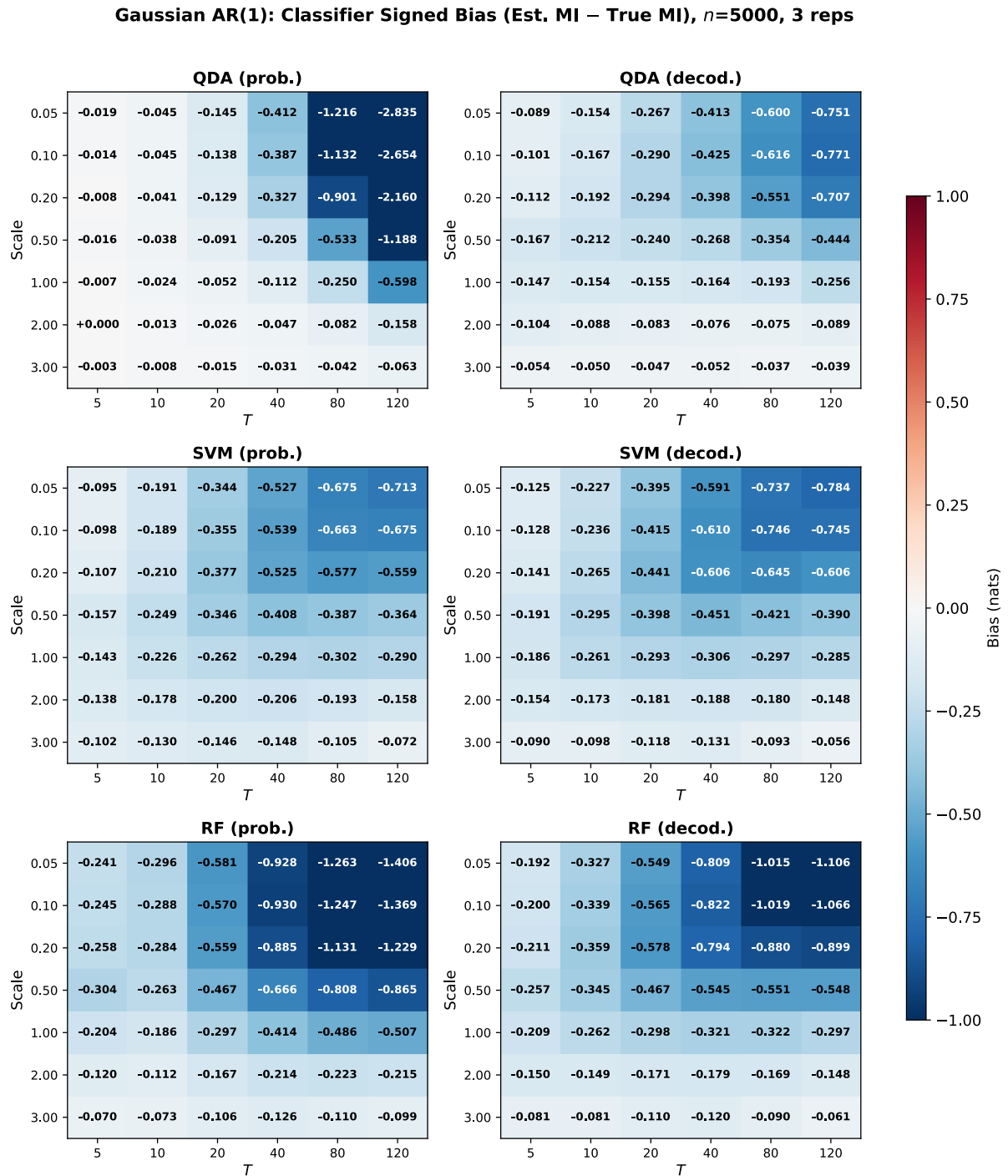


Figure 10: Signed bias for the six classifier-based estimators on Gaussian AR(1) data, $n = 5,000$.

For comparison, Figure 11 shows the corresponding ground-truth mutual information values across the same grid of (λ, T) settings. As expected, the true MI increases with the mean-separation scale λ , since larger class means make the stimulus classes easier to distinguish. It also tends to increase with trajectory length T , because longer trajectories contain more temporal information about the stimulus, before eventually approaching the entropy ceiling $H(S) = \ln 8 \approx 2.08$ nats.

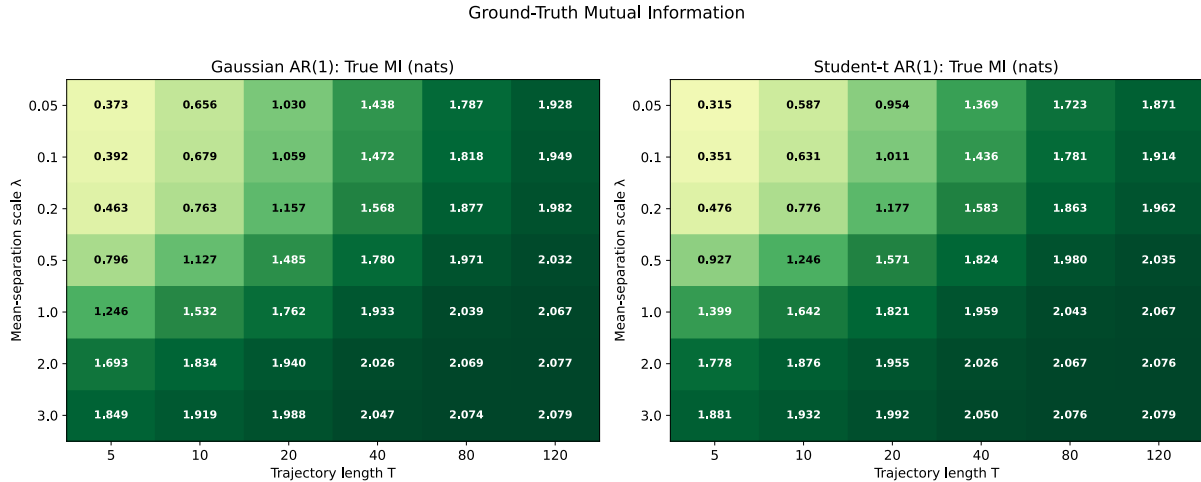


Figure 11: Ground-truth mutual information (in nats) for each combination of mean-separation scale λ and trajectory length T , computed by high-precision Monte Carlo integration.

On Gaussian AR(1) data (Figure 10), all six classifier-based estimators underestimate MI across the grid, but how much they underestimate depends a lot on the choice of classifier and on whether we compute MI from predicted probabilities or from decoded labels. QDA gives the clearest example: because its Gaussian class model matches the distribution of the data, the probability-based QDA estimator is close to unbiased across most of the grid. Its bias stays within roughly 0.05 nats whenever the trajectory is short ($T \leq 10$), regardless of λ , and also whenever the classes are well separated ($\lambda \geq 2$), regardless of T . The estimator only breaks down in the opposite corner, where separation is weak and the trajectory is long: at $\lambda = 0.05$, the bias drops from -0.019 at $T = 5$ to -2.835 at $T = 120$. This is not a failure of the Gaussian assumption itself, which holds exactly here, but of finite-sample estimation, as QDA must estimate a separate $T \times T$ covariance matrix for each of the eight classes, and when those classes are barely distinguishable the resulting density estimates become unreliable. So probability-based QDA can be very accurate when its model is correct and the estimation problem is manageable, but it is also very fragile when the response dimension grows and the classes overlap heavily.

The decoding-based version of QDA is much more stable. It is negatively biased throughout, but its errors stay far smaller than those of the probability-based version in the hardest settings. This gap suggests that using the full predicted probability vector can help when those probabilities are well calibrated, but can make errors much worse when calibration breaks down in high dimensions. We see a similar but less extreme pattern for SVM and RF. Among these two, SVM is the more stable, with moderate negative bias across the grid and only a small difference between its probability-based and decoding-based forms. RF performs noticeably worse when mutual information is low ($\lambda < 0.5$), especially in its probability-based form, where the bias reaches -1.406 at $(0.05, 120)$. This matches the known tendency of random forest probability estimates

to shrink toward the uniform baseline $1/K$, which pushes the MI estimate down.

On Student- t AR(1) data, the overall picture is similar, but the effect of model misspecification stands out much more. Again, all six classifier-based estimators underestimate MI across the grid, with the largest errors at small λ and large T . But the probability-based QDA estimator now fails badly. Its bias reaches -5.317 at $(\lambda, T) = (0.05, 120)$ and stays strongly negative even at intermediate scales. This is a sharp contrast with its strong performance on Gaussian data and shows that the good behaviour of QDA probabilities depends heavily on the Gaussian likelihood model being correct.

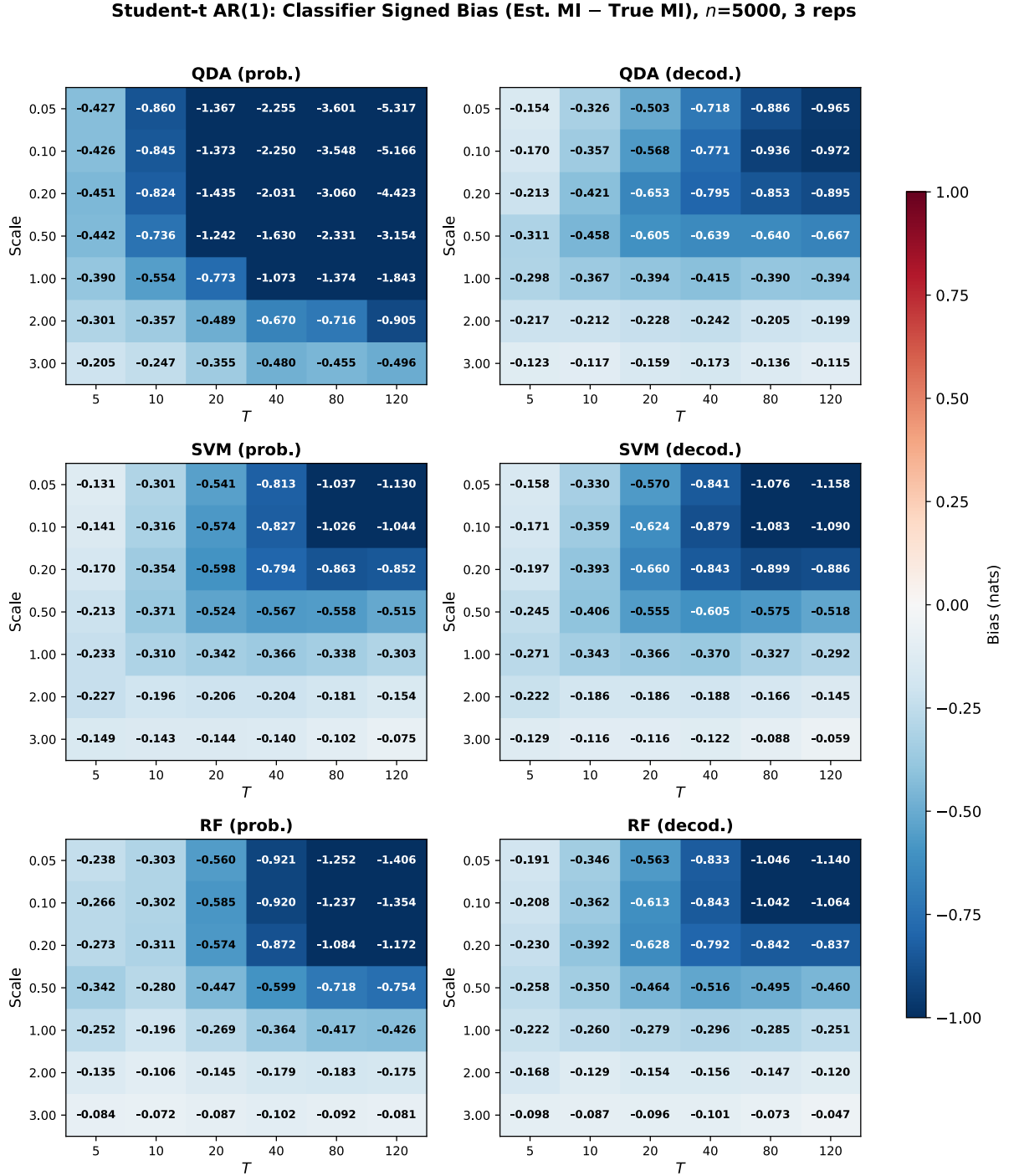


Figure 12: Signed bias for the six classifier-based estimators on Student- t AR(1) data ($\nu = 3$), $n = 5,000$.

The decoding-based QDA estimator is again much more robust, which tells us that although label-only estimation throws away information, it is less exposed to severe model mismatch. SVM and RF are less affected by switching from the Gaussian to Student- t setting, which makes sense given their weaker distributional assumptions. Hence, we conclude that the probability-based estimators can take full advantage of a correct parametric model, but they are also much more sensitive to weak separation, growing dimension, and distributional misspecification.

The neural estimators behave differently on Gaussian data. MINE is the only method that shows noticeable positive bias, and this only happens in the low-information (low- λ) band of the grid. For example, its bias is +0.084 at (0.05, 5) and +0.107 at (0.05, 10). As either λ or T grows, this positive bias goes away and MINE becomes mildly to moderately negatively biased. Compared with the classifier-based estimators, its absolute errors stay relatively small across most of the grid, and it performs better overall. This advantage stands out in the hardest low-separation, high-dimensional regime (top-right corner of the grid). For $T \geq 80$ and $\lambda \leq 0.2$, MINE is the only estimator that does not struggle badly: on Gaussian data its bias ranges from -0.017 to -0.263 , whereas all classifier-based methods are much more negative and VCE is also worse in this region.

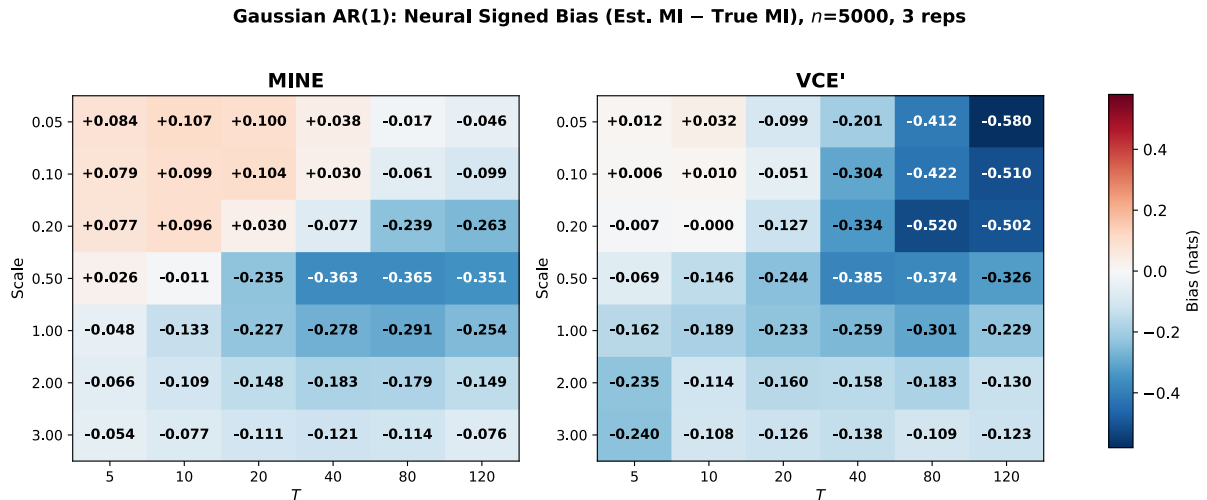


Figure 13: Signed bias for MINE and VCE on Gaussian AR(1) data, $n = 5,000$.

VCE is also competitive, but does not beat MINE in general. An important finding though is that it often beats MINE when both MI and trajectory length are small. Across the region $T < 20$ and $\lambda < 0.2$, VCE has smaller absolute bias in 7 of the 8 Gaussian and Student- t cases we looked at. On Gaussian data, for example, its bias is +0.012 versus +0.084 for MINE at $(\lambda, T) = (0.05, 5)$, and +0.010 versus +0.099 at $(0.1, 10)$. So VCE is a good choice when MI is low and trajectories are short, where it stays closer to unbiased than MINE in most of these cases. But this does not hold as T grows. Outside this region, VCE is usually more negatively biased than MINE, especially at small λ and moderate-to-large T . VCE also has a second weakness that the other methods do not share. Most estimators do worst at low λ and large T , but VCE also underestimates noticeably when the trajectories are very short and the classes are already well separated. At $T = 5$ and $\lambda \geq 2$, for example, its bias sits around -0.24 on Gaussian data, even though this is a high-MI regime where several other estimators do much better. So VCE is unusual in that it underestimates in both the low- λ , large- T corner and the high- λ , small- T corner.

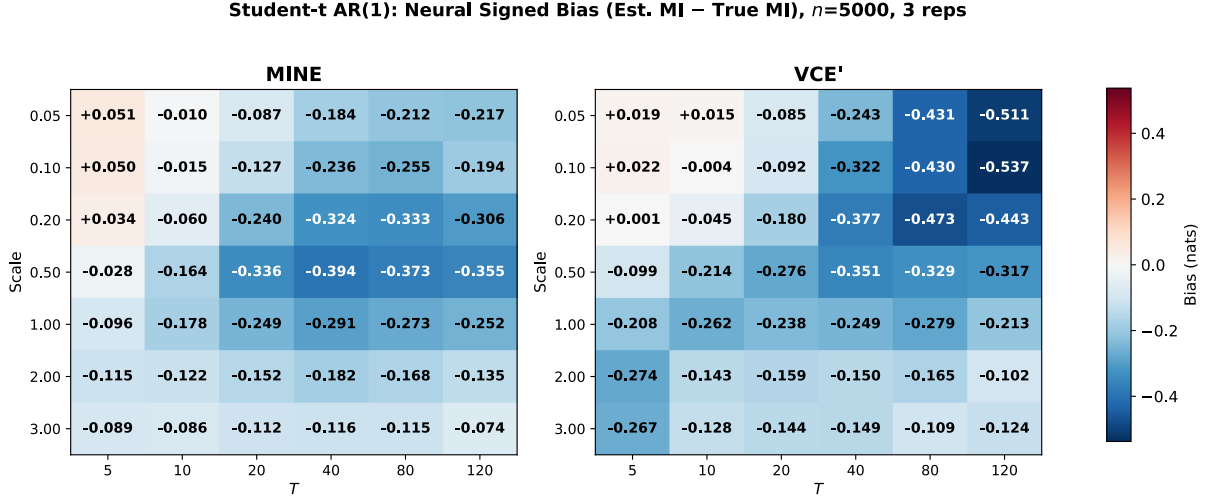


Figure 14: Signed bias for MINE and VCE on Student- t AR(1) data ($\nu = 3$), $n = 5,000$.

On Student- t AR(1) data, both neural estimators behave similarly. MINE again shows only a small region of positive bias at the lowest values of λ and T , and is otherwise negatively biased, though usually not by a lot. The main change from the Gaussian case is that the positive-bias region shrinks, while the rest of the bias surface stays similar. MINE also remains the most robust estimator in the hardest low-separation, high-dimensional regime: for $T \geq 80$ and $\lambda \leq 0.2$, its bias ranges from -0.194 to -0.333 , which is much smaller in magnitude than what we see from the classifier-based estimators and VCE.

VCE also behaves similarly to the Gaussian case. It stays close to unbiased in the low-MI, short-trajectory settings where it often beats MINE, but gets worse as T grows at low λ . We also still see its short-trajectory, high-separation failure. Putting these results together, the neural estimators are clearly more robust to distributional change than probability-based QDA. Between the two, MINE gives the strongest overall performance across the full grid, while VCE is especially useful when mutual information is low and trajectories are short.

Overall, our results show that no single estimator is best in every regime, which is consistent with the No Free Lunch theorem (Wolpert and Macready, 1997). In matched Gaussian settings with well-separated classes, the probability-based QDA estimator can be very accurate, but this good performance is fragile and breaks down under weak separation and long trajectories or model misspecification. The decoding-based estimators are more stable across regimes, with QDA-decoding giving the strongest performance overall and SVM close behind. Among the neural methods, VCE is often the better choice when class separation is low and trajectories are short, while MINE is the most robust method overall and the only estimator that stays reliable in the hardest setting with low mutual information and long trajectories.

8 Conclusion

In this project, we studied several machine learning methods for estimating mutual information as a way of quantifying information transfer in biological signalling. Our main interest was in the setting where a stimulus acts on a cell and produces a response that is observed as a trajectory over time, so that the problem is not only noisy but also high-dimensional, making the estimation of mutual information particularly difficult. To examine this problem under controlled conditions, we worked with synthetic stimulus-response trajectories generated from class-dependent AR(1) models with both Gaussian and Student- t innovations, which allowed the true mutual information to be approximated to high precision by Monte Carlo methods.

We first considered the probability-based method, which estimates mutual information from the probabilities predicted by the classifier. We then studied the decoding-based method, which uses only the predicted labels through the confusion matrix. The comparison showed that when the predicted probabilities were reliable, the probability-based estimator could keep information that decoding threw away. But when those probabilities were poor, the decoding-based estimator was often the more stable option.

We then turned to two neural methods. MINE directly optimises a lower bound on MI using a learned neural network, while VCE first learns each marginal distribution separately through flow matching, maps the data to uniform vector ranks, and then runs MINE in that rank space to estimate the copula density. Across our experiments, MINE turned out to be the most robust estimator overall: it is the only method that still works reasonably well in the hardest setting with low mutual information and long response trajectories, where the other estimators all become clearly more biased. VCE also has a second weakness, as it underestimates at short trajectory lengths with high class separation, where the problem should be easy. That said, VCE does outperform MINE in one important region: when the trajectories are short and the class separation is low, VCE stays closer to unbiased than MINE in most of the cases we tested, which makes it the better pick when the data are low-dimensional and MI is small.

Looking back at the objectives in Section 1.1, I implemented four MI estimators (probability-based SLEMI, the decoding-based approach, MINE, and VCE) and ran all four on the same synthetic AR(1) signalling data, under both Gaussian and Student- t ($\nu = 3$) innovations. The 7×6 grid of mean-separation scales and trajectory lengths, repeated for both data families, gives a clear picture of how each estimator’s bias depends on the two factors we were most interested in: the separation between stimulus classes, which controls how much information is available, and the number of time steps, which controls the dimensionality of the response. From this grid, the relative strengths and limitations of the two families of estimators stand out clearly. Classifier-based estimators are accurate when their assumptions match the data, but can be fragile under weak separation, long trajectories, or distributional misspecification, especially probability-based QDA, which collapses in the Student- t setting. Neural estimators are more robust to all three, though VCE has its own weakness at short trajectory lengths with high separation. On the question of when the extra flexibility of neural estimators actually pays off, the answer is mostly in the hardest regimes: when the classes are already well separated, a good classifier is just as effective. Moreover, when we have very few samples, classifier-based methods are likely to be the more practical choice, since they typically need less data to train reliably than

neural networks, though this was not part of our experimental grid.

Looking forward, there are a few directions worth exploring. Since our comparison was entirely on synthetic data, the most pressing question is how these estimators perform on real experimental trajectories from a well-studied signalling pathway where MI estimates from other methods already exist. It would also be useful to test how each method behaves as the sample size is reduced, since this was held fixed throughout our experiments, and to extend the comparison to settings with a larger number of stimulus classes.

A Mutual Information as Vector Copula Entropy

In the continuous multivariate setting of [Chen et al. \(2025\)](#), mutual information can be written as the negative differential entropy of the vector copula density:

$$I(X; Y) = -H[c(u_X, u_Y)], \quad (44)$$

where u_X and u_Y are the vector ranks associated with X and Y , and $c(u_X, u_Y)$ is the corresponding vector copula density. This relies on the density form of the vector Sklar theorem from [Fan and Henry \(2023\)](#), namely

$$p(x, y) = p(x) p(y) c(u_X, u_Y). \quad (45)$$

The first step is to show that the joint density of the vector ranks is exactly the vector copula density. By definition of vector rank, the transformations $x \mapsto u_X$ and $y \mapsto u_Y$ send the marginals of X and Y to multivariate uniforms on their respective unit hypercubes. Hence

$$p(u_X) = 1, \quad p(u_Y) = 1. \quad (46)$$

Equivalently, by the change-of-variables formula,

$$p(u_X) = |J_{xu_X}|^{-1} p(x), \quad p(u_Y) = |J_{yu_Y}|^{-1} p(y), \quad (47)$$

where J_{xu_X} and J_{yu_Y} denote the Jacobians of the maps from x to u_X and from y to u_Y , respectively. Applying the same change-of-variables formula to the joint density gives

$$p(u_X, u_Y) = |J_{xu_X}|^{-1} |J_{yu_Y}|^{-1} p(x, y). \quad (48)$$

Substituting the vector-Sklar factorisation yields

$$\begin{aligned} p(u_X, u_Y) &= |J_{xu_X}|^{-1} |J_{yu_Y}|^{-1} p(x) p(y) c(u_X, u_Y) \\ &= p(u_X) p(u_Y) c(u_X, u_Y) \\ &= c(u_X, u_Y). \end{aligned} \quad (49)$$

So the vector copula density is exactly the joint density of the vector ranks. Now, under the continuous multivariate assumptions of [Chen et al. \(2025\)](#), namely that the marginals are absolutely continuous with support in convex sets, so that the vector-rank maps exist and are almost-everywhere invertible, mutual information is invariant under this transformation:

$$I(X; Y) = I(u_X; u_Y). \quad (50)$$

Using the entropy representation of mutual information,

$$I(u_X; u_Y) = H(u_X) + H(u_Y) - H(u_X, u_Y). \quad (51)$$

Since u_X and u_Y are uniform on unit hypercubes, their differential entropies are both zero:

$$H(u_X) = 0, \quad H(u_Y) = 0. \quad (52)$$

Therefore,

$$I(X; Y) = -H(u_X, u_Y). \quad (53)$$

Finally, by (49), the joint density of (u_X, u_Y) is $c(u_X, u_Y)$, so

$$H(u_X, u_Y) = H[c(u_X, u_Y)]. \quad (54)$$

Substituting this gives

$$I(X; Y) = -H[c(u_X, u_Y)], \quad (55)$$

which is the required result.

References

- David Barber and Felix V. Agakov. The IM algorithm: A variational approach to information maximization. In *Advances in Neural Information Processing Systems 16*. MIT Press, 2003.
- Mohamed Ishmael Belghazi, Aristide Baratin, Devansh Rajeshwar, Sherjil Ozair, Yoshua Bengio, Aaron Courville, and R. Devon Hjelm. Mutual information neural estimation. In Jennifer Dy and Andreas Krause, editors, *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 531–540. PMLR, 2018.
- Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.
- Peter J. Brockwell and Richard A. Davis. *Introduction to Time Series and Forecasting*. Springer, 3 edition, 2016. ISBN 978-3-319-29852-8.
- Sarah Anhala Cepeda-Humerez, Jakob Ruess, and Gašper Tkačik. Estimating information in time-varying signals. *PLOS Computational Biology*, 15(9):e1007290, 2019. doi: 10.1371/journal.pcbi.1007290.
- Yanzhi Chen, Zijing Ou, Adrian Weller, and Michael U. Gutmann. Neural mutual information estimation with vector copulas. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- Victor Chernozhukov, Alfred Galichon, Marc Hallin, and Marc Henry. Monge-kantorovich depth, quantiles, ranks and signs. *The Annals of Statistics*, 45(1):223–256, 2017.
- Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. doi: 10.1007/BF00994018.
- Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley-Interscience, 1 edition, 1991. ISBN 0-471-06259-6.
- Francis H. C. Crick. Central dogma of molecular biology. *Nature*, 227(5258):561–563, 1970. doi: 10.1038/227561a0.
- Monroe D. Donsker and S. R. S. Varadhan. Asymptotic evaluation of certain Markov process expectations for large time, IV. *Communications on Pure and Applied Mathematics*, 36(2):183–212, 1983. doi: 10.1002/cpa.3160360204.
- Yanqin Fan and Marc Henry. Vector copulas. *Journal of Econometrics*, 234(1):128–150, 2023. doi: 10.1016/j.jeconom.2021.11.012.
- Alejandro A. Granados, Julian M. J. Pietsch, Sarah A. Cepeda-Humerez, Iseabail L. Farquhar, Gašper Tkačik, and Peter S. Swain. Distributed and dynamic intracellular organization of extracellular information. *Proceedings of the National Academy of Sciences*, 115(23):6088–6093, 2018. doi: 10.1073/pnas.1716659115.
- Michael Gutmann and Aapo Hyvärinen. Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. In *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, pages 297–304. JMLR Workshop and Conference Proceedings, 2010.

- Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer, New York, 2 edition, 2009. ISBN 978-0-387-84857-0.
- Kurt Hornik, Maxwell Stinchcombe, and Halbert White. Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366, 1989.
- Tomasz Jetka, Karol Nieniałowski, Tomasz Winarski, Sławomir Błoński, and Michał Komorowski. Information-theoretic analysis of multivariate single-cell signaling responses. *PLOS Computational Biology*, 15(7):e1007132, 2019. doi: 10.1371/journal.pcbi.1007132.
- Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matt Le. Flow matching for generative modeling. *arXiv preprint arXiv:2210.02747*, 2023.
- Jian Ma and Zengqi Sun. Mutual information is copula entropy. *Tsinghua Science & Technology*, 16(1):51–54, 2011.
- Roger B. Nelsen. *An Introduction to Copulas*. Springer Series in Statistics. Springer, 2 edition, 2006.
- John C. Platt. Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In Alexander J. Smola, Peter L. Bartlett, Bernhard Schölkopf, and Dale Schuurmans, editors, *Advances in Large Margin Classifiers*, pages 61–74. MIT Press, Cambridge, MA, 1999.
- Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.
- Akash Srivastava, Seungwook Han, Kai Xu, Benjamin Rhodes, and Michael U. Gutmann. Estimating the density ratio between distributions with high discrepancy using multinomial logistic regression. *Transactions on Machine Learning Research*, 2023. URL <https://openreview.net/forum?id=jM8nzUzBWr>.
- Ying Tang and Alexander Hoffmann. Quantifying information of intracellular signaling: progress with machine learning. *Reports on Progress in Physics*, 85(8):086601, 2022. doi: 10.1088/1361-6633/ac7a4a.
- David H. Wolpert and William G. Macready. No free lunch theorems for optimization. *IEEE Transactions on Evolutionary Computation*, 1(1):67–82, April 1997. doi: 10.1109/4235.585893.